HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

CHUYÊN ĐỀ HỆ THỐNG THÔNG TIN

Đề tài: Hệ thống Chatbot tư vấn tuyển sinh

Giảng viên hướng dẫn: PGS.TS. Phạm Văn Cường

Nhóm học phần: 02

Nhóm bài tập lớn: 07

Họ và tên sinh viên	Mã sinh viên
Trần Đức Dũng	B22DCCN139
Lê Đức Thiện	B22DCCN823
Trịnh Lê Xuân Bách	B22DCCN056

Hà Nội – 2025

LỜI CẢM ƠN

Để hoàn thành bài tập lớn môn học *"Chuyên đề hệ thống thông tin"* với đề tài *"Hệ thống Chatbot tư vấn tuyển sinh"*, nhóm chúng em đã nhận được sự hướng dẫn, giúp đỡ và động viên nhiệt tình từ nhiều cá nhân và tập thể.

Trước hết, chúng em xin trân trọng gửi lời cảm ơn sâu sắc nhất tới thầy Phạm Văn Cường, giảng viên hướng dẫn của chúng em. Thầy đã tận tình chỉ bảo, định hướng nghiên cứu, cung cấp những kiến thức chuyên môn quý báu và giải đáp mọi thắc mắc của chúng em trong suốt quá trình thực hiện đề tài. Sự nhiệt huyết và phương pháp làm việc khoa học của thầy là nguồn động lực lớn để chúng em vượt qua những khó khăn và hoàn thành tốt nhiệm vụ của mình.

Chúng em cũng xin gửi lời cảm ơn chân thành đến Ban Giám đốc Học viện Công nghệ Bưu chính Viễn thông, Ban chủ nhiệm Khoa Công nghệ Thông tin 1 đã tạo điều kiện học tập và nghiên cứu tốt nhất cho chúng em trong những năm học vừa qua.

Mặc dù đã rất cố gắng, nhưng do kiến thức và kinh nghiệm thực tiễn còn hạn chế, bài báo cáo không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Trần Đức Dũng

Lê Đức Thiện

Trịnh Lê Xuân Bách

MỤC LỤC

LỜI CẢM ƠN i

DANH MỤC HÌNH ẢNH v

THUẬT NGỮ VIẾT TẮT vi

LỜI MỞ ĐẦU 1

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 3

1.1. Đặt vấn đề 3

1.2. Lý do chọn đề tài 3

1.3. Mục tiêu đề tài 4

1.4. Phạm vi đề tài 5

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 7

2.1. Tổng quan về Large Language Models 7

2.1.1. Khái niệm và kiến trúc Transformer 7

2.1.2. Mô hình ngôn ngữ Gemini 2.5 Flash 9

2.2. Kỹ thuật Retrieval-Augmented Generation 10

2.2.1. Khái niệm và sự cần thiết của RAG 10

2.2.2. Quy trình RAG truyền thống 10

2.2.3. Giải pháp RAG nâng cao với LangGraph 11

2.3. Cơ chế lưu trữ và truy xuất nâng cao 12

2.3.1. Mô hình nhúng ngôn ngữ - Word Embeddings 12

2.3.2. Cơ sở dữ liệu Véc-tơ với ChromaDB 13

2.3.3. Chiến lược ParentDocumentRetriever - Hierarchical Indexing 13

2.4. Framework và công cụ phát triển 14

2.4.1. Thư viện LangChain 14

2.4.2. Giao diện người dùng với Streamlit 15

2.4.3. Kỹ thuật thu thập dữ liệu - Web Scraping 16

2.4.4. Định dạng dữ liệu 17
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 19
3.1. Phân tích yêu cầu hệ thống 19
3.1.1. Yêu cầu chức năng 19
3.1.2. Yêu cầu phi chức năng 19
3.2. Kiến trúc tổng thể hệ thống 20
3.2.1. Chi tiết các thành phần cốt lõi 21
3.2.2. Luồng đi của một câu hỏi qua hệ thống 22
3.3. Quy trình xây dựng và tiền xử lý dữ liệu 23
3.3.1. Thu thập và làm sạch dữ liệu 23
3.3.2. Thiết kế chiến lược phân mảnh phân cấp 24
3.3.3. Lưu trữ Vector và Docstore 26
3.4. Thiết kế luồng điều phối RAG nâng cao 27
3.4.1. Quản lý trạng thái và bộ nhớ hội thoại 28
3.4.2. Nút phân tích truy vấn 29
3.4.3. Nút truy xuất song song 29
3.4.4. Nút sinh câu trả lời 30
3.4.5. Đồ thị điều phối - Graph Routing 31
3.5. Thiết kế giao diện và tương tác 32
3.5.1. Xây dựng giao diện hội thoại 32
3.5.2. Thiết kế cơ chế Human-in-the-Loop 33
CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ 35
4.1. Môi trường và công cụ triển khai 35
4.1.1. Cấu hình phần cứng và phần mềm 35
4.1.2. Thiết lập kết nối API và biến môi trường 37

4.2. Kết quả thu thập và xử lý dữ liệu.....38
4.2.1. Thống kê dữ liệu thu thập.....39
4.2.2. Thống kê kết quả phân mảnh.....39
4.3. Trình diễn các chức năng cốt lõi.....39
4.3.1. Giao diện trang chủ và tương tác cơ bản.....39
4.3.2. Kịch bản hội thoại đa lượt.....40
4.3.3. Kịch bản bóc tách câu hỏi phức tạp.....40
4.3.4. Kịch bản chủ động hỏi lại.....41
4.4. Đánh giá hiệu năng và chất lượng hệ thống.....42
4.4.1. Đánh giá tốc độ phản hồi.....42
4.4.2. Đánh giá độ chính xác.....43
4.5. Hạn chế và hướng phát triển.....44
4.5.1. Các hạn chế hiện tại.....44
4.5.2. Hướng phát triển mở rộng.....45
KẾT LUẬN CHUNG.....47
TÀI LIỆU THAM KHẢO.....49

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ kiến trúc tổng quan của mô hình Transformer.....8

Hình 2.2: Minh họa cơ chế Self-Attention giúp mô hình liên kết và hiểu ngữ cảnh văn bản.....9

Hình 2.3: Sơ đồ luồng hoạt động tuyến tính của hệ thống RAG truyền thống.....11

Hình 2.4: Minh họa chiến lược ParentDocumentRetriever giúp cân bằng giữa độ chính xác khi tìm kiếm và sự toàn vẹn của ngữ cảnh.....14

Hình 2.5: Cấu trúc giao diện hội thoại cơ bản xây dựng bằng Streamlit.....16

Hình 2.6: Mô hình tổng quan của quá trình Web Scraping bóc tách dữ liệu từ website. 17

Hình 3.1: Sơ đồ khối kiến trúc tổng thể của hệ thống Chatbot RAG nâng cao.....21

Hình 3.2: Minh họa chiến lược phân mảnh phân cấp: Tìm kiếm trên Child Chunks và truy xuất ngữ cảnh từ Parent Chunks.....26

Hình 3.3: Giao diện hội thoại chính của chatbot tư vấn tuyển sinh xây dựng bằng Streamlit.....33

Hình 4.1: Cấu hình môi trường ảo và danh sách các thư viện được cài đặt trong dự án. 36

Hình 4.2: Cấu hình mã khóa trên Google AI Studio.....37

Hình 4.2: Giao diện quản lý và khởi tạo API Key trên nền tảng Google AI Studio.....38

Hình 4.3: Cấu hình bảo mật các khóa API trong tệp môi trường .env.....38

Hình 4.4: Giao diện chính của hệ thống và phản hồi cho truy vấn cơ bản.....40

Hình 4.5: Cơ chế Human-in-the-Loop chủ động yêu cầu người dùng làm rõ thông tin41

THUẬT NGỮ VIẾT TẮT

Thuật ngữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
CoT	Chain of Thought	Chuỗi suy luận
CPU	Central Processing Unit	Bộ vi xử lý trung tâm
CSS	Cascading Style Sheets	Ngôn ngữ định dạng giao diện trang web
DOM	Document Object Model	Cấu trúc cây đối tượng của tài liệu HTML
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Định dạng dữ liệu có cấu trúc dạng cặp khóa-giá trị
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng nơ-ron bộ nhớ ngắn hạn dài
MCP	Model Context Protocol	Giao thức ngữ cảnh mô hình (Kết nối Agent với dữ liệu)
PDF	Portable Document Format	Định dạng tài liệu văn bản di động
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy

RPM	Requests Per Minute	Số lượng yêu cầu trên mỗi phút
URL	Uniform Resource Locator	Đường dẫn tài nguyên

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, sự phát triển bùng nổ của trí tuệ nhân tạo và các mô hình ngôn ngữ lớn đang tạo ra những bước ngoặt đột phá trong mọi mặt của đời sống. Đặc biệt trong lĩnh vực giáo dục và đào tạo, việc chuyển đổi số và ứng dụng công nghệ thông minh không chỉ là xu hướng tất yếu mà còn là thước đo cho sự đổi mới của các cơ sở giáo dục. Các hệ thống tương tác tự động hóa, tiêu biểu là chatbot thông minh, đang dần khẳng định vai trò quan trọng trong việc nâng cao chất lượng dịch vụ, tối ưu hóa quy trình vận hành và mang lại trải nghiệm tiện ích vượt trội cho người sử dụng.

Hàng năm, mỗi kỳ tuyển sinh đại học đến luôn kéo theo một khối lượng thông tin khổng lồ cần được truyền tải. Các bạn học sinh THPT và quý phụ huynh thường xuyên phải đối mặt với vô vàn thắc mắc xoay quanh ngành học, chỉ tiêu, điểm chuẩn, phương thức xét tuyển hay các vấn đề về học phí. Đối với một cơ sở đào tạo trọng điểm và đi đầu về công nghệ như Học viện Công nghệ Bưu chính Viễn thông, lưu lượng truy cập và số lượng câu hỏi đổ về qua các kênh truyền thông trong giai đoạn này là cực kỳ lớn. Các phương thức hỗ trợ truyền thống, dù được vận hành bởi đội ngũ tư vấn viên tận tâm, đôi khi vẫn gặp những giới hạn nhất định về thời gian phản hồi, khó đáp ứng trọn vẹn tiêu chí "tức thời, liên tục 24/7" mà kỷ nguyên thông tin đòi hỏi.

Xuất phát từ thực tiễn đó, đề tài *"Hệ thống Chatbot tư vấn tuyển sinh"* được thực hiện nhằm mang đến một giải pháp công nghệ hiện đại và toàn diện. Khác với các mô hình sinh ngôn ngữ thông thường dễ mắc phải hiện tượng "ảo giác", dự án tập trung phát triển một hệ thống hội thoại thông minh dựa trên kiến trúc RAG nâng cao kết hợp luồng điều phối linh hoạt từ LangGraph. Bằng cách tối ưu hóa năng lực xử lý của mô hình ngôn ngữ lớn, hệ thống được thiết kế thành một đặc vụ định tuyến có khả năng tự động bóc tách các câu hỏi phức tạp thành nhiều ý nhỏ để truy xuất song song, đồng thời áp dụng chiến lược lưu trữ phân cấp để đảm bảo ngữ cảnh cung cấp luôn nguyên vẹn và chính xác. Đặc biệt, chatbot còn được trang bị cơ chế giao tiếp hai chiều "Human-in-the-loop", cho phép hệ thống chủ động tạm ngưng và đặt câu hỏi ngược lại khi người dùng cung cấp thiếu dữ kiện. Việc ứng dụng đề tài này kỳ vọng sẽ không chỉ san sẻ áp lực cho bộ phận tuyển sinh, đảm bảo độ tin cậy tuyệt đối về mặt thông tin, mà còn góp

phần xây dựng hình ảnh một môi trường học thuật tiên tiến, luôn sẵn sàng đồng hành cùng thí sinh.

Cấu trúc bài báo cáo bao gồm 4 chương chính:

- Chương 1: Giới thiệu đề tài
- Chương 2: Cơ sở lý thuyết
- Chương 3: Phân tích và thiết kế hệ thống
- Chương 4: Triển khai và đánh giá kết quả

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

Công tác tư vấn tuyển sinh tại Học viện Công nghệ Bưu chính Viễn thông hàng năm luôn phải tiếp nhận một lượng khổng lồ các thắc mắc từ thí sinh và phụ huynh. Các câu hỏi xoay quanh quy chế xét tuyển, chỉ tiêu, điểm chuẩn hay học phí thường có tính lặp lại cao nhưng lại đổ về dồn dập trong một khoảng thời gian ngắn. Điều này không chỉ gây quá tải cho đội ngũ tư vấn viên mà còn hạn chế khả năng hỗ trợ liên tục 24/7, dẫn đến việc chậm trễ trong quá trình cung cấp thông tin quan trọng mang tính quyết định đến việc chọn trường của thí sinh.

Để giải quyết vấn đề này, việc ứng dụng AI và các mô hình ngôn ngữ lớn là một hướng đi tất yếu. Tuy nhiên, các chatbot AI thông thường thường gặp phải hiện tượng "ảo giác thông tin" - tự bịa đặt câu trả lời khi thiếu dữ liệu thực tế, gây rủi ro cực kỳ lớn trong lĩnh vực tư vấn tuyển sinh vốn đòi hỏi độ chính xác tuyệt đối. Ngay cả khi áp dụng kỹ thuật RAG truyền thống, hệ thống vẫn bộc lộ sự cứng nhắc: thiếu khả năng tự đánh giá độ liên quan của tài liệu tìm kiếm được, không xử lý tốt các câu hỏi phức tạp nhiều vế, và không biết chủ động hỏi lại khi thông tin người dùng cung cấp bị mơ hồ.

Từ những hạn chế trong vận hành thực tiễn và rào cản về ảo giác của công nghệ AI, nhu cầu cấp thiết là xây dựng một hệ thống tư vấn thông minh, có tính chủ động cao. Việc ứng dụng kiến trúc RAG nâng cao dựa trên luồng điều phối của LangGraph chính là giải pháp tối ưu cho Chatbot tuyển sinh PTIT. Hệ thống không chỉ dừng lại ở việc truy xuất thông tin thuần túy mà còn sở hữu khả năng định tuyến thông minh, tự động phân tách các câu hỏi phức tạp để xử lý song song và chủ động tương tác làm rõ dữ kiện khi cần thiết. Sự kết hợp giữa mô hình ngôn ngữ thế hệ mới và chiến lược lưu trữ phân cấp giúp hệ thống vận hành với tốc độ vượt trội, đảm bảo tính minh bạch và độ tin cậy tối đa cho thí sinh.

1.2. Lý do chọn đề tài

Lý do đầu tiên xuất phát từ yêu cầu cấp thiết về việc hiện đại hóa quy trình hỗ trợ tuyển sinh tại Học viện Công nghệ Bưu chính Viễn thông. Trong bối cảnh số lượng thí sinh quan tâm đến các ngành công nghệ ngày càng tăng cao, việc duy trì các phương

thức tư vấn truyền thống qua điện thoại hay tin nhắn thủ công đã bộc lộ những hạn chế về thời gian phản hồi và khả năng chịu tải. Một hệ thống chatbot tự động có khả năng hoạt động không ngừng nghỉ 24/7 sẽ là lời giải hoàn hảo để đồng hành cùng thí sinh mọi lúc, mọi nơi, đồng thời giúp nhà trường tối ưu hóa nguồn lực nhân sự trong những giai đoạn cao điểm.

Bên cạnh đó, việc lựa chọn đề tài này còn bắt nguồn từ mong muốn ứng dụng những bước tiến mới nhất của trí tuệ nhân tạo vào thực tế. Dù công nghệ RAG truyền thống đã chứng minh được hiệu quả trong việc giảm thiểu sai lệch thông tin, nhưng thực tế triển khai cho thấy các luồng xử lý tuyến tính đơn giản vẫn chưa đủ linh hoạt để giải quyết những câu hỏi phức tạp hoặc thiếu dữ kiện. Việc nghiên cứu và áp dụng kiến trúc RAG nâng cao thông qua khung làm việc LangGraph cho phép hệ thống sở hữu khả năng định tuyến thông minh, phân tách truy vấn để xử lý song song và chủ động tương tác làm rõ thông tin với người dùng. Điều này không chỉ giúp đảm bảo tính chính xác và đầy đủ của thông tin mà còn mang lại trải nghiệm tương tác tự nhiên, nhạy bén như một tư vấn viên thực thụ.

Cuối cùng, dự án là cơ hội để hiện thực hóa một sản phẩm phần mềm có giá trị ứng dụng cao, dựa trên sự kết hợp giữa kho dữ liệu thực tế của Học viện và các công nghệ hàng đầu như mô hình Gemini 2.5 Flash, ChromaDB và mô hình nhúng chuyên biệt cho tiếng việt. Việc xây dựng thành công chatbot này không chỉ mang ý nghĩa về mặt kỹ thuật trong việc tối ưu hóa tốc độ phản hồi và độ chính xác, mà còn góp phần khẳng định vị thế dẫn đầu về công nghệ của PTIT thông qua việc ứng dụng AI vào ngay chính công tác quản lý và hỗ trợ thí sinh, sinh viên của nhà trường.

1.3. Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng thành công một hệ thống Chatbot tư vấn tuyển sinh thông minh dành riêng cho Học viện Công nghệ Bưu chính Viễn thông. Hệ thống này không chỉ đóng vai trò là một kho lưu trữ thông tin trực tuyến mà còn phải là một trợ lý ảo đáng tin cậy, có khả năng giao tiếp tự nhiên và cung cấp thông tin chính xác tuyệt đối dựa trên các nguồn tài liệu chính thống của nhà trường, từ đó hỗ trợ đắc lực cho thí sinh và phụ huynh trong quá trình tìm hiểu về kỳ tuyển sinh.

Về mặt kỹ thuật, đề tài hướng tới việc làm chủ và triển khai thành công kiến trúc Agentic RAG tiên tiến. Mục tiêu cụ thể là thiết kế một luồng xử lý dữ liệu hoàn chỉnh: từ khâu tự động thu thập dữ liệu từ website ptit.edu.vn, đến việc xử lý và lưu trữ dữ liệu phân cấp nhằm tối ưu hóa khả năng truy xuất. Hệ thống phải thực hiện được các tác vụ thông minh như: tự động phân tích và chia nhỏ câu hỏi phức tạp thành các tiểu mục, đánh giá độ liên quan của tài liệu để loại bỏ nhiễu.

Cuối cùng, dự án đặt mục tiêu hoàn thiện một ứng dụng thực tế với giao diện trực quan trên nền tảng Web, đảm bảo tính ổn định và tốc độ phản hồi nhanh. Hệ thống cần tích hợp được khả năng ghi nhớ lịch sử hội thoại và cơ chế tương tác xác thực với người dùng khi gặp các yêu cầu mơ hồ. Qua đó, đề tài không chỉ dừng lại ở mức độ nghiên cứu lý thuyết mà còn tạo ra một công cụ có khả năng triển khai thực tiễn, đóng góp vào công cuộc chuyển đổi số trong giáo dục của Học viện.

1.4. Phạm vi đề tài

Về mặt nội dung và dữ liệu, đề tài tập trung vào việc thu thập và xử lý các thông tin liên quan trực tiếp đến công tác tuyển sinh của Học viện Công nghệ Bưu chính Viễn thông. Dữ liệu đầu vào giới hạn trong các nguồn chính thống từ website chính thức của nhà trường và các văn bản quy chế tuyển sinh hiện hành. Các thông tin này bao gồm: chỉ tiêu tuyển sinh, các phương thức xét tuyển, điểm chuẩn các năm, thông tin chi tiết về các ngành đào tạo và các quy định về học phí, học bổng. Đề tài không mở rộng sang các mảng dữ liệu nằm ngoài phạm vi tuyển sinh như quản lý đào tạo chuyên sâu hay các hoạt động hành chính khác của Học viện.

Về mặt đối tượng người dùng, hệ thống được thiết kế để phục vụ chủ yếu cho thí sinh đang chuẩn bị bước vào kỳ thi đại học và các bậc phụ huynh có nhu cầu tìm hiểu thông tin về nhà trường. Do đó, ngôn ngữ và phong cách phản hồi của chatbot sẽ được tối ưu hóa cho tiếng Việt, đảm bảo tính gần gũi, chính xác và dễ hiểu đối với đại đa số người dùng phổ thông.

Về mặt công nghệ, dự án tập trung triển khai mô hình RAG nâng cao dựa trên khung điều phối của LangGraph, kết hợp mô hình ngôn ngữ Gemini 2.5 Flash và mô hình nhúng chuyên biệt cho tiếng Việt. Hệ thống được vận hành trên nền tảng Streamlit, tập trung vào việc tối ưu hóa luồng định tuyến thông minh, bóc tách câu hỏi phức tạp để

truy xuất song song và áp dụng cơ chế Human-in-the-loop nhằm chủ động làm rõ thông tin từ thí sinh. Phạm vi nghiên cứu giới hạn ở việc xử lý dữ liệu văn bản đa cấu trúc Markdown, JSON, PDF từ các nguồn dữ liệu chính thống của Học viện, ưu tiên tính chính xác và tốc độ phản hồi thực tế thay vì xây dựng một hạ tầng quy mô lớn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Large Language Models

2.1.1. Khái niệm và kiến trúc Transformer

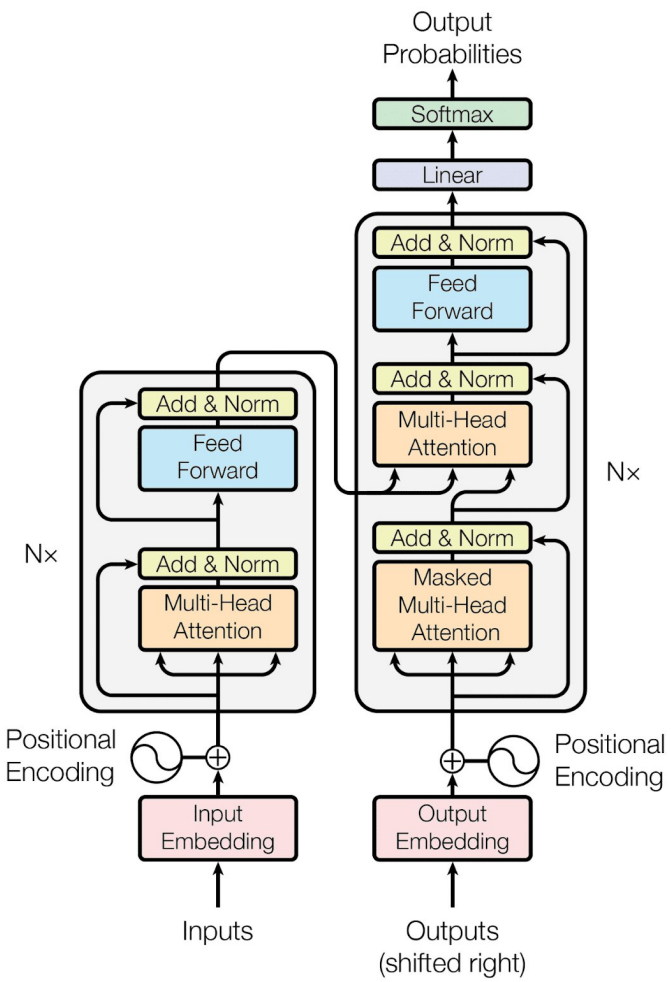
2.1.1.1 Khái niệm mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn là các hệ thống học sâu được huấn luyện trên các tập dữ liệu văn bản khổng lồ với số lượng tham số lên đến hàng tỷ. Mục tiêu cốt lõi của các mô hình này là hiểu, phân tích và sinh ra ngôn ngữ tự nhiên của con người. Về mặt toán học, bản chất hoạt động của một LLM là quá trình tính toán xác suất để dự đoán từ hoặc token tiếp theo sẽ xuất hiện trong một chuỗi văn bản, dựa trên ngữ cảnh của các từ đã xuất hiện trước đó. Nhờ khả năng "học" được các quy luật ngữ pháp, ngữ nghĩa và cả kiến thức thế giới từ dữ liệu huấn luyện, LLM hiện đại có thể thực hiện tốt nhiều tác vụ phức tạp như dịch thuật, tóm tắt văn bản, viết mã nguồn và đặc biệt là hỏi đáp một cách trôi chảy.

2.1.1.2. Kiến trúc Transformer - Nền tảng của LLMs

Đột phá thực sự tạo nên kỷ nguyên của các LLM hiện đại chính là kiến trúc Transformer, được nhóm nghiên cứu của Google giới thiệu lần đầu vào năm 2017 trong bài báo khoa học nổi tiếng *"Attention Is All You Need"*. Trước khi Transformer ra đời, các bài toán xử lý chuỗi thời gian hay ngôn ngữ thường phụ thuộc vào các mạng nơ-ron hồi quy RNN hoặc LSTM. Những mạng này có nhược điểm lớn là xử lý dữ liệu tuần tự, dẫn đến tốc độ huấn luyện chậm và dễ bị "quên" thông tin khi đối mặt với các đoạn văn bản dài - vấn đề vanishing gradient.

Kiến trúc Transformer đã giải quyết hoàn toàn rào cản này bằng cách loại bỏ cơ chế xử lý tuần tự và cho phép tính toán song song trên toàn bộ câu.

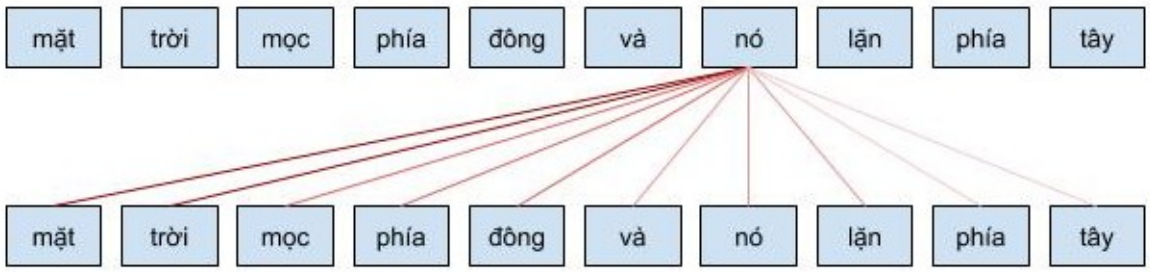


Hình 2.1: Sơ đồ kiến trúc tổng quan của mô hình Transformer.

2.1.1.3. Cơ chế tự chú ý

Trái tim làm nên sức mạnh của kiến trúc Transformer chính là cơ chế Self-Attention. Khác với việc chỉ nhìn vào từ ngay sát trước nó, cơ chế Self-Attention cho phép mô hình khi xử lý một từ cụ thể có thể "nhìn" vào tất cả các từ khác trong cùng một câu để đánh giá mức độ liên quan giữa chúng, bất kể khoảng cách xa gần.

Điều này giúp mô hình nắm bắt được các mối liên hệ phụ thuộc xa và hiểu được ngữ cảnh đa nghĩa một cách sâu sắc. Ví dụ: Trong hệ thống chatbot tuyển sinh, nếu người dùng đặt câu hỏi: *"Điểm chuẩn của ngành An toàn thông tin năm ngoái là bao nhiêu và học phí của nó thế nào?"*, cơ chế Self-Attention sẽ giúp mô hình tự động hiểu rằng đại từ *"nó"* ở cuối câu có trọng số liên kết mạnh nhất với cụm *"ngành An toàn thông tin"* ở đầu câu, từ đó truy xuất đúng thông tin cần thiết.



Hình 2.2: Minh họa cơ chế Self-Attention giúp mô hình liên kết và hiểu ngữ cảnh văn bản.

2.1.2. Mô hình ngôn ngữ Gemini 2.5 Flash

2.1.2.1. Giới thiệu về mô hình Gemini 2.5 Flash

Trong dự án này, mô hình ngôn ngữ lớn được lựa chọn làm "bộ não" trung tâm điều phối toàn bộ hệ thống chatbot là Gemini 2.5 Flash. Đây là dòng mô hình thế hệ mới của Google, được thiết kế tối ưu cho tốc độ phản hồi cực nhanh mà vẫn duy trì năng lực suy luận mạnh mẽ. Với khả năng hiểu và xử lý tiếng việt tự nhiên, mô hình có thể nắm bắt chính xác các thuật ngữ tuyển sinh đặc thù và các cách diễn đạt đa dạng của thí sinh.

Việc lựa chọn Gemini 2.5 Flash thay vì các mô hình mã nguồn mở chạy cục bộ là một quyết định chiến lược để đáp ứng các tác vụ phức tạp trong kiến trúc RAG nâng cao. Trong hệ thống sử dụng LangGraph, LLM không chỉ đơn thuần là bộ phận sinh văn bản mà đóng vai trò là một Router Agent thực hiện các nhiệm vụ đòi hỏi tư duy logic:

- Phân tích ý định: Nhận diện chính xác câu hỏi thuộc lĩnh vực tuyển sinh hay giao tiếp thông thường để điều phối luồng xử lý.
- Chia nhỏ truy vấn: Tự động bóc tách một câu hỏi phức tạp thành nhiều câu hỏi con để tìm kiếm dữ liệu đa chiều.
- Quyết định làm rõ: Đóng vai trò kiểm soát chất lượng đầu vào, nhận diện các câu hỏi thiếu dữ kiện để kích hoạt cơ chế hỏi lại người dùng.

2.1.2.2. Tối ưu hiệu năng và xử lý ngữ cảnh khổng lồ

Một trong những thách thức lớn nhất của các hệ thống RAG nâng cao là "độ trễ" khi phải thực hiện nhiều bước xử lý và "tràn ngữ cảnh" khi nhồi nhét quá nhiều tài liệu. Gemini 2.5 Flash giải quyết triệt để các nút thắt này thông qua hai đặc tính kỹ thuật vượt trội:

- Tốc độ xử lý thời gian thực: Thông qua API của Google AI Studio, Gemini 2.5 Flash cho phép thực hiện các bước phân tích định tuyến và tổng hợp tài liệu chỉ trong vài giây. Điều này giúp hệ thống LangGraph có thể chạy luồng truy xuất song song và phản hồi kết quả gần như tức thời lên giao diện Streamlit, mang lại trải nghiệm mượt mà cho người dùng.
- Cửa sổ ngữ cảnh lên tới 1 triệu token: Khác với các mô hình bị giới hạn về bộ nhớ, Gemini 2.5 Flash có khả năng đọc và hiểu khối lượng tài liệu cực lớn. Điều này đặc biệt quan trọng khi kết hợp với chiến lược ParentDocumentRetriever của dự án, nơi AI cần tiếp nhận rất nhiều đoạn văn bản dài từ kho tri thức của Học viện để trích xuất câu trả lời mà không lo bị mất mát thông tin hay lỗi vượt quá giới hạn bộ nhớ.

2.2. Kỹ thuật Retrieval-Augmented Generation

2.2.1. Khái niệm và sự cần thiết của RAG

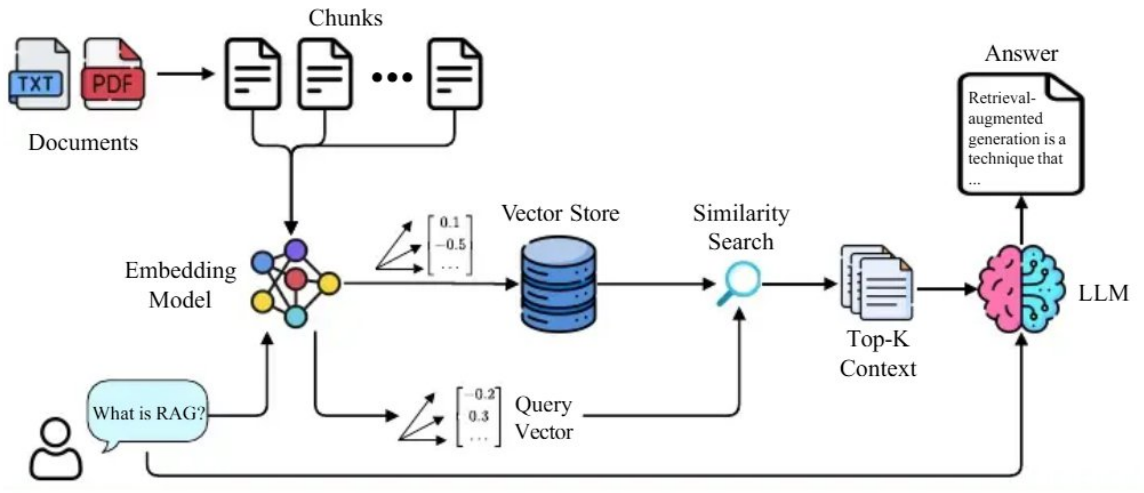
Mặc dù các mô hình ngôn ngữ lớn sở hữu khả năng ngôn ngữ vượt trội, chúng vẫn tồn tại hai hạn chế chí mạng khi áp dụng vào các bài toán thực tế mang tính đặc thù: (1) Hiện tượng ảo giác thông tin - mô hình sinh ra các thông tin nghe có vẻ hợp lý nhưng thực chất là sai lệch hoặc không có thật; và (2) Sự thiếu hụt kiến thức cập nhật/nội bộ - do mô hình bị đóng băng kiến thức tại thời điểm huấn luyện, nó không thể biết được các quy chế tuyển sinh mới nhất của Học viện Công nghệ Bưu chính Viễn thông.

Để khắc phục vấn đề này, kỹ thuật RAG - Sinh văn bản tăng cường truy xuất đã ra đời. Thay vì dựa hoàn toàn vào "trí nhớ" - tham số nội tại của LLM để trả lời, RAG kết nối mô hình ngôn ngữ với một cơ sở dữ liệu tri thức bên ngoài. Khi người dùng đặt câu hỏi, hệ thống sẽ ưu tiên tìm kiếm các tài liệu thực tế có liên quan nhất từ kho dữ liệu, sau đó cung cấp đoạn tài liệu này làm "ngữ cảnh" - Context để ép LLM sinh ra câu trả lời dựa trên những thông tin đã được kiểm chứng. Điều này giúp hệ thống tư vấn tuyển sinh đảm bảo tính chính xác tuyệt đối, ngăn chặn việc chatbot tự ý bịa ra điểm chuẩn hay học phí.

2.2.2. Quy trình RAG truyền thống

Một hệ thống RAG cơ bản hoạt động theo một luồng xử lý tuyến tính bao gồm 3 giai đoạn chính:

1. Lập chỉ mục (Indexing): Dữ liệu dạng văn bản ví dụ: các file Markdown crawl từ website PTIT được chia nhỏ thành các đoạn ngắn (chunks). Các đoạn này sau đó được chuyển đổi thành các véc-tơ số học thông qua mô hình Embedding và lưu trữ vào cơ sở dữ liệu Véc-tơ (Vector Database).
2. Truy xuất (Retrieval): Khi có câu hỏi từ người dùng, hệ thống chuyển câu hỏi đó thành véc-tơ và thực hiện tìm kiếm tính tương đồng (thường dùng Cosine Similarity) trong Vector Database để lấy ra các đoạn tài liệu (chunks) gần nghĩa nhất.
3. Sinh văn bản (Generation): Các đoạn tài liệu lấy được sẽ được ghép nối vào câu lệnh prompt cùng với câu hỏi gốc của người dùng, sau đó gửi đến LLM để tổng hợp thành câu trả lời cuối cùng.



Hình 2.3: Sơ đồ luồng hoạt động tuyến tính của hệ thống RAG truyền thống.

Tuy nhiên, cấu trúc tuyến tính này bộc lộ nhiều điểm yếu: nếu câu hỏi của người dùng mơ hồ hoặc chứa nhiều vấn đề đan xen, hệ thống dễ truy xuất dữ liệu nhiễu hoặc không trúng trọng tâm. LLM ở bước cuối sẽ thụ động trả lời dựa trên thông tin thiếu sót mà không có cơ chế phân tách yêu cầu hay chủ động làm rõ dữ kiện, dẫn đến trải nghiệm tương tác rời rạc và thiếu chính xác.

2.2.3. Giải pháp RAG nâng cao với LangGraph

Để vượt qua giới hạn của RAG truyền thống, dự án áp dụng kiến trúc RAG nâng cao. Trong kiến trúc này, hệ thống không vận hành theo một chuỗi cố định mà được thiết kế như một đồ thị trạng thái có khả năng định tuyến thông minh và xử lý song song thông qua khung làm việc LangGraph.

Sự khác biệt cốt lõi của hệ thống thể hiện qua các mô-đun điều phối tối ưu:

- Định tuyến và phân tách truy vấn: Hệ thống đóng vai trò một đặc vụ định tuyến, có khả năng bóc tách các câu hỏi phức tạp thành nhiều truy vấn phụ độc lập để bao quát toàn diện thông tin.
- Truy xuất song song: Sử dụng kỹ thuật Fan-out (Send API) để thực hiện tìm kiếm đồng thời trên nhiều luồng dữ liệu. Điều này giúp tối ưu hóa tốc độ phản hồi và đảm bảo tính đa chiều của thông tin trích xuất.
- Tương tác chủ động (Human-in-the-Loop): Đây là điểm nhấn thông minh của hệ thống. Nếu câu hỏi thiếu dữ kiện bắt buộc, đồ thị sẽ chủ động tạm dừng và yêu cầu người dùng làm rõ, tránh việc AI đưa ra câu trả lời dựa trên sự suy đoán.
- Quản lý ngữ cảnh qua MemorySaver: Thông qua thread_id, LangGraph duy trì trạng thái hội thoại liên tục, giúp chatbot hiểu được các đại từ nhân xưng và kế thừa thông tin từ các lượt chat trước đó một cách tự nhiên.

2.3. Cơ chế lưu trữ và truy xuất nâng cao

2.3.1. Mô hình nhúng ngôn ngữ - Word Embeddings

Để máy tính có thể so sánh ý nghĩa văn bản, dữ liệu cần được chuyển đổi thành các véc-tơ số học trong không gian đa chiều sao cho những đoạn văn bản có ngữ nghĩa tương đồng sẽ nằm gần nhau. Trong dự án này, mô hình nhúng được sử dụng là dangvantuan/vietnamese-embedding thay vì các mô hình đa ngôn ngữ thông thường.

Việc lựa chọn mô hình này mang lại hai lợi thế cốt lõi:

1. Độ chính xác ngữ nghĩa vượt trội: Đây là mô hình được huấn luyện tinh chỉnh chuyên biệt trên kho dữ liệu tiếng Việt khổng lồ. Nhờ đó, hệ thống có khả năng phân biệt cực tốt các thuật ngữ tuyển sinh đặc thù (như "xét tuyển kết hợp", "hệ chính quy") và hiểu đúng các từ viết tắt của PTIT mà các mô hình đa ngôn ngữ thường bỏ sót.
2. Không gian biểu diễn chi tiết: Với kích thước véc-tơ 768 chiều, mô hình cung cấp khả năng biểu diễn thông tin chi tiết và sắc nét hơn. Điều này giúp ChromaDB thực hiện tìm kiếm tương đồng chính xác tuyệt đối, đảm bảo dữ liệu cung cấp cho mô hình Gemini luôn đúng trọng tâm và giàu ngữ cảnh.

2.3.2. Cơ sở dữ liệu Véc-tơ với ChromaDB

Sau khi các tài liệu được chuyển đổi thành véc-tơ, chúng cần một hệ quản trị cơ sở dữ liệu chuyên biệt để lưu trữ và truy vấn hiệu quả. Hệ thống sử dụng ChromaDB – một Vector Database mã nguồn mở, tối ưu cho các ứng dụng AI.

Khi hệ thống LangGraph nhận được câu hỏi từ người dùng ví dụ: "Ngành đa phương tiện xét tuyển khối nào?", câu hỏi này cũng sẽ được mô hình nhúng chuyển thành một véc-tơ truy vấn. ChromaDB sau đó sẽ thực hiện thuật toán tìm kiếm tương đồng thường là Cosine Similarity để tính toán khoảng cách giữa véc-tơ truy vấn và các véc-tơ tài liệu đã lưu, từ đó trả về các đoạn văn bản gần nghĩa nhất. Sự tích hợp liền mạch giữa ChromaDB và LangChain giúp việc thiết lập luồng truy xuất trở nên nhanh chóng và dễ dàng mở rộng.

2.3.3. Chiến lược ParentDocumentRetriever - Hierarchical Indexing

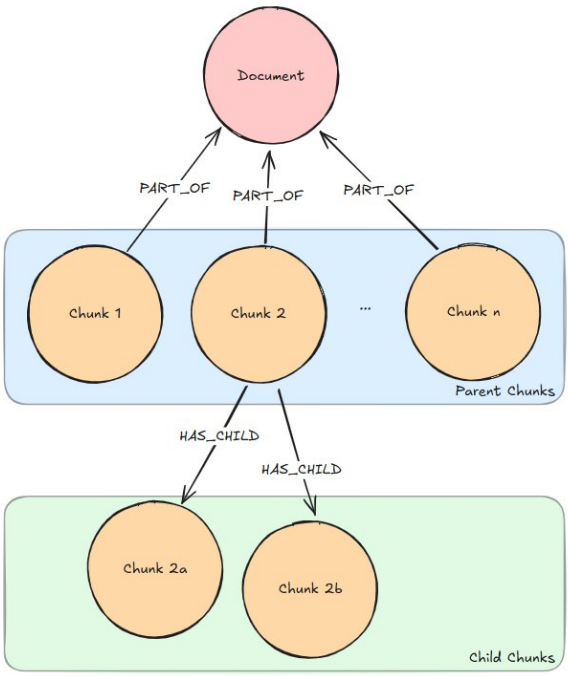
Một trong những thách thức lớn nhất của kỹ thuật RAG là việc chọn kích thước phân mảnh tài liệu - chunk size.

- Nếu chia chunk quá lớn (ví dụ: cả một trang web), việc tìm kiếm véc-tơ sẽ kém chính xác vì ý nghĩa bị pha loãng.
- Nếu chia chunk quá nhỏ (ví dụ: từng câu một), việc tìm kiếm sẽ cực kỳ chính xác, nhưng LLM lại thiếu đi ngữ cảnh xung quanh để hiểu trọn vẹn vấn đề, dẫn đến câu trả lời chắp vá.

Để giải quyết triệt để bài toán đánh đổi này, dự án áp dụng chiến lược ParentDocumentRetriever - Truy xuất tài liệu phân cấp của LangChain. Kỹ thuật này tách biệt quá trình "tìm kiếm" và quá trình "cung cấp ngữ cảnh" bằng cách tạo ra hai loại chunk khác nhau:

1. Child Chunks - Phân mảnh con: Tài liệu gốc được chia thành các đoạn nhỏ, quy mô khoảng 300 ký tự. Các đoạn nhỏ này chứa các ý nghĩa rất cô đọng và từ khóa cụ thể (ví dụ: chỉ một dòng về mức học phí). Các Child chunks được nhúng và lưu trữ trong ChromaDB để phục vụ việc tìm kiếm chính xác.
2. Parent Chunks - Phân mảnh cha: Cùng một tài liệu đó được chia thành các đoạn lớn hơn, quy mô khoảng 1500 ký tự, bao trùm toàn bộ ngữ cảnh (ví dụ: cả một mục lớn về Quy định học phí và học bổng). Các Parent chunks này được lưu trữ riêng trong một kho chứa tài liệu nội bộ - Docstore.

Cơ chế hoạt động: Khi có truy vấn, hệ thống sẽ rà quét trong ChromaDB để tìm ra các Child chunks có độ tương đồng cao nhất. Tuy nhiên, thay vì gửi trực tiếp các Child chunks ngắn ngủi này cho LLM, hệ thống sẽ dùng chúng làm "chìa khóa" để tra ngược lại Docstore, lấy ra toàn bộ Parent chunks tương ứng chứa chúng.



Hình 2.4: Minh họa chiến lược ParentDocumentRetriever giúp cân bằng giữa độ chính xác khi tìm kiếm và sự toàn vẹn của ngữ cảnh.

Nhờ chiến lược Hierarchical Indexing này, mô hình Gemini 2.5 Flash luôn nhận được những khối thông tin đầy đủ, mạch lạc, giúp sinh ra câu trả lời tự nhiên, trọn vẹn mà không bỏ sót các điều kiện đi kèm trong quy chế tuyển sinh của nhà trường.

2.4. Framework và công cụ phát triển

2.4.1. Thư viện LangChain

LangChain là framework mã nguồn mở đóng vai trò "xương sống" để kết nối các thành phần của hệ thống. Trong dự án này, LangChain chuẩn hóa giao tiếp giữa mô hình ngôn ngữ Gemini 2.5 Flash, cơ sở dữ liệu véc-tơ ChromaDB và kho tài liệu tuyển sinh của Học viện.

Dự án đã khai thác các mô-đun cốt lõi của LangChain bao gồm:

- Document Loaders & Text Splitters: Sử dụng MarkdownHeaderTextSplitter và RecursiveCharacterTextSplitter để chia nhỏ văn bản dựa trên cấu trúc tiêu đề.

Điều này giúp giữ lại tính phân cấp của tài liệu (ví dụ: thông tin học phí nằm dưới đúng tên ngành tương ứng), hỗ trợ AI hiểu đúng ngữ cảnh.

- Retrievers (ParentDocumentRetriever): LangChain cung cấp giải pháp truy xuất tài liệu phân cấp chuyên sâu. Thành phần này cho phép hệ thống tìm kiếm véc-tơ trên các mảnh nhỏ (Child chunks) để đạt độ chính xác cao, nhưng khi trả về sẽ lấy các đoạn văn bản lớn hơn (Parent chunks) để đảm bảo AI có đủ thông tin toàn diện để trả lời.
- Prompt Templates & Output Parsers: Quản lý các mẫu câu lệnh và chuẩn hóa dữ liệu đầu ra dưới dạng JSON, giúp nút định tuyến analyze_query hoạt động ổn định.

LangChain cung cấp các "viên gạch" kỹ thuật nền tảng, tạo tiền đề để thư viện LangGraph xây dựng lên trên và điều phối thành một luồng RAG nâng cao có khả năng định tuyến thông minh và xử lý song song.

2.4.2. Giao diện người dùng với Streamlit

Streamlit là một framework mã nguồn mở dựa trên ngôn ngữ Python, được thiết kế chuyên biệt để xây dựng và triển khai các ứng dụng web tương tác cho các dự án dữ liệu và trí tuệ nhân tạo. Ưu điểm vượt trội của Streamlit là cho phép các nhà phát triển tạo ra giao diện người dùng trực quan, hiện đại một cách nhanh chóng mà không cần tốn nhiều thời gian xử lý các ngôn ngữ lập trình front-end phức tạp như HTML, CSS hay JavaScript.

Hình 2.5: Cấu trúc giao diện hội thoại cơ bản xây dựng bằng Streamlit.

Trong dự án chatbot tư vấn tuyển sinh PTIT, Streamlit không chỉ là màn hình hiển thị đơn thuần mà còn đảm nhiệm các chức năng tương tác cốt lõi:

- Xây dựng Giao diện Hội thoại: Streamlit cung cấp sẵn các thành phần giao diện chuyên dụng cho chatbot như st.chat_message, st.chat_input, giúp hiển thị rành mạch lịch sử nhắn tin giữa thí sinh và trợ lý AI.
- Quản lý Phiên làm việc: Streamlit quản lý các biến trạng thái của người dùng trong suốt phiên truy cập. Bằng cách kết hợp tính năng này với thread_id của LangGraph, hệ thống có thể lưu trữ và ghi nhớ lịch sử hội thoại, giúp chatbot hiểu được ngữ cảnh của các câu hỏi nối tiếp nhau.
- Xử lý cơ chế Human-in-the-Loop: Đây là điểm giao tiếp quan trọng nhất giữa giao diện và luồng xử lý. Khi đồ thị LangGraph tạm dừng tại node human_clarify do câu hỏi của người dùng bị thiếu thông tin (ví dụ: *"Ngành này học phí bao nhiêu?"* nhưng không rõ là ngành nào), Streamlit sẽ bắt tín hiệu này để hiển thị các yêu cầu làm rõ, nhận thông tin bổ sung từ người dùng và gửi phản hồi trở lại để LangGraph tiếp tục luồng xử lý.

2.4.3. Kỹ thuật thu thập dữ liệu - Web Scraping

Để hệ thống RAG nâng cao có thể cung cấp các câu trả lời chính xác và không bị "ảo giác", nó cần một cơ sở tri thức đáng tin cậy. Do các thông tin tuyển sinh mới nhất thường được cập nhật trực tiếp trên website của Học viện dưới dạng các bài viết HTML và không có sẵn API truy xuất, kỹ thuật Web Scraping được áp dụng như bước đầu tiên và mang tính quyết định trong luồng xử lý dữ liệu của dự án.

Hình 2.6: Mô hình tổng quan của quá trình Web Scraping bóc tách dữ liệu từ website.

Trong module crawl/ của dự án, bộ công cụ thu thập dữ liệu được xây dựng dựa trên các thư viện Python chuyên dụng:

- Requests và BeautifulSoup: Thư viện requests đảm nhiệm việc gửi các giao thức HTTP để tải về toàn bộ mã nguồn HTML của trang web mục tiêu. Ngay sau đó, BeautifulSoup được sử dụng để phân tích cú pháp cây cấu trúc DOM. Công cụ này cho phép định vị và bóc tách chính xác các đoạn văn bản hữu ích (nằm trong các thẻ tiêu đề <h1>, <h2>, thẻ đoạn văn <p> hoặc bảng biểu <table>), đồng thời tự động lọc bỏ các thành phần nhiễu không mang giá trị tư vấn như thanh điều hướng, chân trang hay các banner quảng cáo.
- Framework Scrapy: Đối với các nhiệm vụ thu thập quy mô lớn, đòi hỏi phải tự động duyệt qua nhiều trang liên kết hoặc đệ quy theo các đường dẫn nội bộ để lấy toàn bộ bài viết trong chuyên mục tuyển sinh, Scrapy được sử dụng như một bộ khung tự động hóa mạnh mẽ. Scrapy không chỉ giúp quản lý luồng cào dữ liệu hiệu quả, kiểm soát tốc độ gửi yêu cầu để không làm ảnh hưởng đến máy chủ của trường, mà còn hỗ trợ xuất dữ liệu thô một cách có hệ thống.

Khối lượng dữ liệu thô thu thập được từ bước này chính là nguyên liệu đầu vào quan trọng, chuẩn bị cho quá trình tiền xử lý và định dạng ở các bước tiếp theo trước khi đưa vào cơ sở dữ liệu Véc-tơ.

2.4.4. Định dạng dữ liệu

Sau quá trình thu thập dữ liệu thô, việc chuẩn hóa và lựa chọn định dạng lưu trữ (crawled_data/) đóng vai trò quyết định đến chất lượng đầu vào của hệ thống RAG. Khác với con người có thể đọc hiểu văn bản từ nhiều nguồn với các định dạng hiển thị phức tạp, các mô hình ngôn ngữ lớn cần dữ liệu được cấu trúc hóa để tối ưu hiệu năng truy xuất. Dự án kết hợp sử dụng ba định dạng chính yếu:

- Markdown (.md) - Định dạng chủ đạo: Đây là định dạng quan trọng nhất trong hệ thống RAG. Các tài liệu HTML sau khi cào về sẽ được chuyển đổi sang Markdown. Lý do ưu tiên định dạng này là vì nó duy trì được cấu trúc phân cấp nội dung cực kỳ trực quan thông qua các ký hiệu tiêu đề (#, ##, ###), danh sách (*, -) và bảng biểu. Điều này tương thích hoàn hảo với công cụ MarkdownHeaderTextSplitter của LangChain, cho phép hệ thống chia nhỏ văn

bản mà vẫn tự động gắn kèm các "đường dẫn ngữ cảnh" - breadcrumbs vào mỗi đoạn nhỏ (ví dụ: [Tuyển sinh 2024 > Phương thức xét tuyển > Xét tuyển kết hợp]). Nhờ đó, LLM không bao giờ bị mất đi ngữ cảnh gốc của thông tin.

- JSON - Quản lý dữ liệu có cấu trúc: Định dạng này được sử dụng để lưu trữ các thông tin có tính định lượng và phân lớp rõ ràng, ví dụ như: bảng mã ngành, chỉ tiêu tuyển sinh từng ngành, các tổ hợp môn xét tuyển hoặc danh sách các câu hỏi thường gặp. Việc sử dụng cặp khóa - giá trị (Key-Value) của JSON giúp hệ thống dễ dàng truy xuất các thông tin đặc thù một cách chính xác tuyệt đối mà không cần LLM phải suy luận quá nhiều.
- PDF - Xử lý văn bản pháp quy chính thức: rất nhiều thông tin quan trọng của Học viện như đề án tuyển sinh chi tiết hay quy chế đào tạo được ban hành dưới dạng file PDF gốc. Hệ thống tích hợp các công cụ phân tích cú pháp PDF (như PyPDFLoader trong LangChain) để trích xuất văn bản từ các tệp này, sau đó đưa vào luồng tiền xử lý chung để làm phong phú thêm cơ sở tri thức cho chatbot.

Sự kết hợp linh hoạt giữa ba định dạng này đảm bảo hệ thống vừa giữ được độ chính xác của các dữ liệu có cấu trúc, vừa tận dụng được sức mạnh tổng hợp ngữ cảnh của tài liệu văn bản, tạo nền tảng dữ liệu vững chắc cho các bước truy xuất và sinh câu trả lời phía sau.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Quá trình phân tích yêu cầu đóng vai trò nền tảng để định hình kiến trúc và các tính năng cốt lõi của chatbot. Dựa trên bài toán tư vấn tuyển sinh thực tế tại PTIT và những điểm yếu của các mô hình AI hiện tại, hệ thống được thiết kế với các yêu cầu nghiêm ngặt sau đây:

3.1.1. Yêu cầu chức năng

Đây là các tính năng bắt buộc mà hệ thống phải thực hiện được để đảm bảo luồng nghiệp vụ tư vấn tuyển sinh diễn ra trơn tru và thông minh:

- Hỗ trợ hội thoại đa lượt: Hệ thống phải có khả năng lưu trữ và theo dõi trạng thái của cuộc trò chuyện thông qua thread_id. Điều này giúp chatbot hiểu được ngữ cảnh của các câu hỏi tiếp nối. Ví dụ: Nếu thí sinh hỏi *"Điểm chuẩn ngành An toàn thông tin là bao nhiêu?"* và sau đó hỏi tiếp *"Học phí của ngành này thế nào?"*, hệ thống phải tự động hiểu đại từ "ngành này" là An toàn thông tin.
- Truy xuất thông tin tuyển sinh nội bộ tự động: Khi nhận được câu hỏi, hệ thống phải tự động phân tích và tìm kiếm thông tin tương ứng trong kho dữ liệu (ChromaDB và Docstore) đã được thu thập từ website PTIT, thay vì sử dụng kiến thức chung chung của mô hình LLM.
- Tự động phân tích và chia nhỏ câu hỏi: Với những câu hỏi phức tạp chứa nhiều ý (ví dụ: *"Ngành Công nghệ thông tin và Đa phương tiện xét khối nào, chỉ tiêu bao nhiêu?"*), hệ thống phải có chức năng bóc tách thành các truy vấn nhỏ để tìm kiếm song song, đảm bảo không bỏ sót ý của người dùng.
- Cơ chế chủ động làm rõ thông tin: Nếu câu hỏi của người dùng quá ngắn hoặc mơ hồ (ví dụ: *"Cho em hỏi về điểm chuẩn"* nhưng không nói rõ ngành nào, cơ sở nào), hệ thống không được phép đoán mò. Thay vào đó, luồng xử lý phải tạm dừng và chủ động đặt câu hỏi ngược lại để người dùng bổ sung thông tin trước khi tiếp tục tìm kiếm.

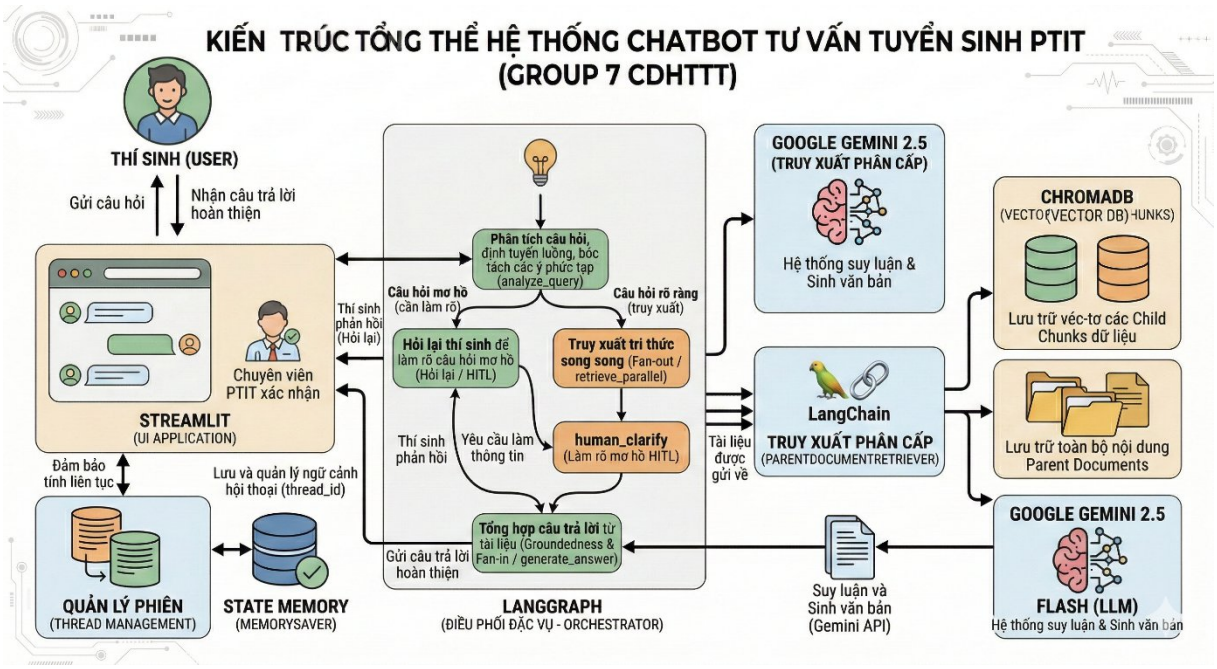
3.1.2. Yêu cầu phi chức năng

Bên cạnh các tính năng cụ thể, hệ thống phải đáp ứng các tiêu chuẩn về chất lượng, hiệu năng và trải nghiệm người dùng:

- Độ chính xác tuyệt đối: Đây là yêu cầu tối quan trọng đối với một chatbot tư vấn giáo dục. Câu trả lời sinh ra phải bám sát 100% vào tài liệu thu thập được từ nhà trường. Nếu sau các vòng lặp tự sửa sai mà vẫn không tìm thấy thông tin, hệ thống phải thẳng thắn trả lời *"Tôi không có thông tin về vấn đề này"* thay vì tự bịa đặt số liệu.
- Hiệu năng và tốc độ phản hồi: Mặc dù áp dụng kiến trúc RAG nâng cao với các bước xử lý thông minh như định tuyến ý định và bóc tách câu hỏi đa chiều, hệ thống vẫn phải đảm bảo tốc độ phản hồi nhanh để mang lại cảm giác tương tác theo thời gian thực. Yêu cầu này được giải quyết triệt để thông qua việc tích hợp mô hình Gemini 2.5 Flash kết hợp cùng kỹ thuật truy xuất song song của LangGraph. Việc cho phép nhiều luồng tìm kiếm chạy đồng thời giúp giảm thiểu tối đa độ trễ, đảm bảo thời gian trả lời trung bình luôn duy trì ở mức dưới 5 giây.
- Trải nghiệm người dùng và giao diện thân thiện: Giao diện ứng dụng xây dựng qua Streamlit cần được thiết kế trực quan, mô phỏng các ứng dụng nhắn tin quen thuộc. Hệ thống cần hiển thị rõ các trạng thái "đang suy nghĩ", "đang tìm kiếm tài liệu" hoặc "yêu cầu làm rõ" để người dùng không bị bối rối trong quá trình chờ đợi.
- Khả năng mở rộng và cập nhật dữ liệu: Dữ liệu tuyển sinh thay đổi theo từng năm. Do đó, kiến trúc hệ thống phải cho phép quản trị viên dễ dàng thêm mới tài liệu vào thư mục crawled_data/ và chạy lại kịch bản lập chỉ mục mà không cần can thiệp sâu vào mã nguồn cốt lõi của chatbot.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống Chatbot tư vấn tuyển sinh PTIT được thiết kế theo kiến trúc hướng dịch vụ vi mô, trong đó các thành phần được phân tách rõ ràng về mặt chức năng nhưng liên kết chặt chẽ với nhau thông qua luồng dữ liệu. Kiến trúc này bao gồm 3 phân hệ chính: giao diện người dùng, lõi xử lý trí tuệ nhân tạo và cơ sở dữ liệu tri thức.



Hình 3.1: Sơ đồ khối kiến trúc tổng thể của hệ thống Chatbot RAG nâng cao.

3.2.1. Chi tiết các thành phần cốt lõi

1. Tầng giao diện (module app.py): Xây dựng trên nền tảng Streamlit, đóng vai trò quản lý luồng hội thoại và hiển thị nguồn trích dẫn. Tầng này chịu trách nhiệm tiếp nhận câu hỏi và thực hiện cơ chế Human-in-the-loop, cho phép hệ thống tạm dừng đồ thị để yêu cầu người dùng bổ sung dữ kiện trước khi tiến hành truy xuất.
2. Tầng Điều phối (module rag_graph.py): Là trung tâm xử lý dựa trên LangGraph và mô hình Gemini 2.5 Flash. Thay vì luồng xử lý tuần tự, tầng này vận hành một đồ thị định tuyến thông minh giúp phân loại ý định, bóc tách đa truy vấn và sử dụng Send API để kích hoạt các luồng tìm kiếm song song, tối ưu hóa triệt để tốc độ phản hồi.
3. Tầng trí thức (ChromaDB & Docstore): Lưu trữ dữ liệu tuyển sinh đã qua làm sạch và mã hóa bởi mô hình vietnamese-embedding. Hệ thống áp dụng chiến lược ParentDocumentRetriever: sử dụng ChromaDB để tìm kiếm véc-tơ trên các phân mảnh nhỏ nhằm đạt độ chính xác cao, đồng thời kết hợp với Docstore để trích xuất các phân mảnh lớn, đảm bảo cung cấp đầy đủ ngữ cảnh cho mô hình ngôn ngữ.

3.2.2. Luồng đi của một câu hỏi qua hệ thống

Để tối ưu hóa tốc độ phản hồi và độ chính xác, quy trình xử lý một truy vấn của thí sinh trong hệ thống RAG nâng cao được thực hiện qua các bước định tuyến và xử lý song song như sau:

Bước 1: Phân tích và định tuyến (analyze_query): Khi nhận được câu hỏi từ giao diện Streamlit, "bộ não" Gemini 2.5 Flash sẽ tiến hành phân tích đa tầng:

- Phân loại Intent: Xác định câu hỏi thuộc luồng tuyển sinh hay giao tiếp thông thường.
- Kiểm tra độ rõ ràng: Đánh giá xem câu hỏi có thiếu dữ kiện then chốt hay không.
- Decomposition (Phân rã): Nếu câu hỏi phức tạp (hỏi nhiều ý cùng lúc), hệ thống tự động bẻ lái và chia nhỏ thành tối đa 3 truy vấn phụ (sub-queries) để chuẩn bị cho việc tìm kiếm đa chiều.

Bước 2: Cơ chế "Đạp phanh" chủ động (Human-in-the-Loop): Nếu nút phân tích xác định câu hỏi mơ hồ (clarification_needed: true), đồ thị LangGraph sẽ kích hoạt trạng thái tạm dừng (Interrupt). Tại giao diện người dùng, hệ thống hiển thị yêu cầu bổ sung thông tin. Luồng xử lý chỉ được tiếp tục chạy vào cơ sở dữ liệu sau khi thí sinh đã cung cấp đủ dữ kiện cần thiết, giúp loại bỏ hoàn toàn việc AI phải "đoán" ý người dùng.

Bước 3: Truy xuất song song đa luồng (Parallel Retrieval): Đây là bước đột phá về hiệu suất. Thay vì tìm kiếm tuần tự, hệ thống sử dụng kỹ thuật Fan-out thông qua Send API của LangGraph. Mỗi truy vấn phụ được xử lý độc lập và đồng thời trên các luồng khác nhau. Các véc-tơ câu hỏi được quét qua ChromaDB để tìm các phân mảnh nhỏ (Child chunks) có độ tương đồng cao nhất.

Bước 4: Ánh xạ ngữ cảnh phân cấp (Parent Document Retrieval): Từ các "mảnh vỡ" thông tin tìm được ở bước 3, hệ thống thực hiện ánh xạ ngược lại ID trong Docstore để lấy ra các đoạn văn bản mẹ (Parent chunks) trọn vẹn và giàu ngữ cảnh. Kỹ thuật này giúp AI tiếp nhận được đầy đủ các bảng biểu, quy định xung quanh từ khóa mà không bị hiện tượng "đứt gãy thông tin" như các hệ thống RAG truyền thống.

Bước 5: Tổng hợp tri thức và phản hồi: Cuối cùng, hàm _docs_reducer thực hiện gom tụ toàn bộ tài liệu từ các luồng song song về một điểm. LLM sẽ đọc, đối chiếu thông tin và biên soạn câu trả lời cuối cùng. Câu trả lời không chỉ đầy đủ ý mà còn đính kèm

chính xác các nguồn trích dẫn từ website Học viện, đảm bảo tính minh bạch trước khi hiển thị tới người dùng.

3.3. Quy trình xây dựng và tiền xử lý dữ liệu

Chất lượng của bất kỳ hệ thống RAG nào cũng phụ thuộc hoàn toàn vào nguồn dữ liệu đầu vào theo nguyên tắc "Garbage in, Garbage out" (Dữ liệu rác tạo ra kết quả rác). Do đó, việc thu thập, làm sạch và chuẩn hóa dữ liệu từ Cổng thông tin tuyển sinh của Học viện Công nghệ Bưu chính Viễn thông là bước nền tảng vô cùng quan trọng.

3.3.1. Thu thập và làm sạch dữ liệu

3.3.1.1 Quá trình thu thập dữ liệu

Dữ liệu tuyển sinh của Học viện chủ yếu được đăng tải dưới dạng các bài viết tin tức, thông báo HTML và các tệp văn bản đính kèm. Để thu thập toàn bộ khối lượng thông tin này một cách tự động, dự án sử dụng kết hợp các thư viện Python bao gồm requests và BeautifulSoup / Scrapy.

Hệ thống crawler được thiết lập để duyệt qua các chuyên mục trọng điểm như: thông tin tuyển sinh đại học chính quy, các chương trình chất lượng cao, thông báo học phí, và hỏi đáp. Quá trình này tự động quét qua các trang, lần theo các đường dẫn bài viết chi tiết và tải toàn bộ mã nguồn HTML hoặc các file đính kèm về bộ nhớ cục bộ.

3.3.1.2. Làm sạch dữ liệu

Mã nguồn HTML thô thu được chứa rất nhiều thông tin nhiễu không phục vụ cho việc tư vấn, chẳng hạn như: thanh điều hướng, menu, chân trang, các đoạn mã JavaScript, CSS định dạng và các banner quảng cáo. Nếu đưa trực tiếp dữ liệu này vào hệ thống RAG, mô hình ngôn ngữ sẽ bị phân tâm và làm giảm độ chính xác của tìm kiếm.

Bước làm sạch được thực hiện bằng cách sử dụng các bộ lọc của BeautifulSoup để:

- Loại bỏ hoàn toàn các thẻ <script>, <style>, <header>, <footer>, <nav>.
- Chỉ giữ lại phần nội dung cốt lõi nằm trong các thẻ bài viết chính (thường là hoặc các thẻ tương đương).
- Trích xuất các thành phần mang ngữ nghĩa cao như: Tiêu đề (<h1>, <h2>, <h3>), đoạn văn bản (<p>), danh sách (<ul>, <ol>, <li>) và các bảng biểu (<table>).

3.3.1.3. Chuẩn hóa định dạng lưu trữ (.md, .json, .pdf)

Sau khi làm sạch, dữ liệu được chuẩn hóa và lưu trữ vào thư mục crawled_data/ dưới 3 định dạng đặc thù, mỗi định dạng phục vụ một mục đích tối ưu hóa riêng cho LLM:

- Định dạng Markdown: Đây là định dạng chủ đạo của hệ thống. Các bài viết HTML được chuyển đổi sang Markdown để giữ lại nguyên vẹn cấu trúc phân cấp nội dung (ví dụ: thẻ <h1> thành #, <h2> thành ##). Việc chuẩn hóa thành Markdown có ý nghĩa sống còn đối với hệ thống, vì nó cho phép thư viện MarkdownHeaderTextSplitter của LangChain ở bước sau có thể nhận diện các "tiêu đề cha" và gắn ngữ cảnh vào từng đoạn văn bản nhỏ, giúp LLM hiểu đoạn văn đó đang nói về chuyên mục nào.
- Định dạng JSON (.json): Đối với các dữ liệu có cấu trúc dạng bảng biểu như Danh sách mã ngành, Tổ hợp môn xét tuyển, Chỉ tiêu tuyển sinh từng năm, dữ liệu được bóc tách và chuyển đổi thành các file JSON (cặp Key-Value). Việc cấu trúc hóa này giúp loại bỏ sự mơ hồ của ngôn ngữ tự nhiên, đảm bảo khi thí sinh hỏi về "chỉ tiêu ngành An toàn thông tin", hệ thống có thể truy xuất chính xác con số tuyệt đối.
- Định dạng PDF (.pdf): Đối với các văn bản mang tính pháp quy, quy chế chính thức của Học viện như Đề án tuyển sinh chi tiết hàng năm vốn được nhà trường ban hành dưới dạng file PDF, hệ thống crawler sẽ tải trực tiếp các tệp này về lưu trữ. Các tệp PDF này sau đó sẽ được xử lý riêng bằng các module đọc PDF của LangChain nhằm trích xuất văn bản một cách nguyên trạng nhất.

3.3.2. Thiết kế chiến lược phân mảnh phân cấp

3.3.2.1 Bài toán kích thước phân mảnh

Trong hệ thống RAG, sau khi dữ liệu được làm sạch và chuyển sang định dạng Markdown, bước tiếp theo là phải cắt nhỏ văn bản để đưa vào cơ sở dữ liệu véc-tơ. Việc chọn kích thước phân mảnh quyết định trực tiếp đến độ thông minh của chatbot:

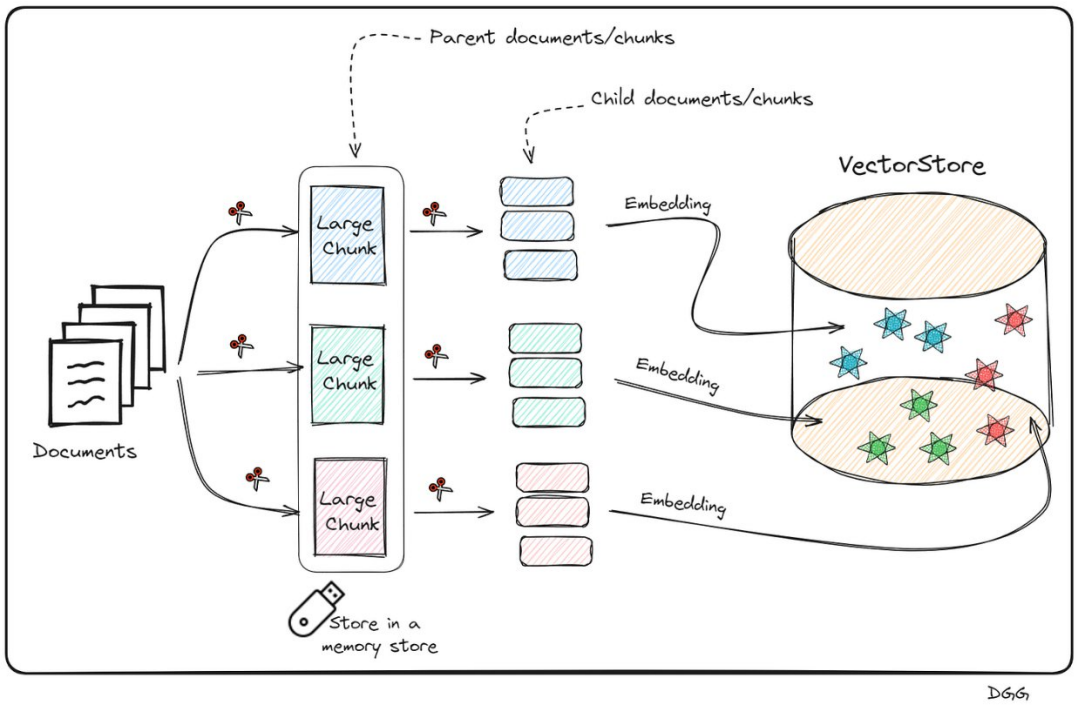
- Nếu cắt quá nhỏ (ví dụ: 1-2 câu): Khả năng tìm kiếm (Retrieval) cực kỳ chính xác vì véc-tơ truy vấn dễ dàng khớp với từ khóa. Tuy nhiên, khi đưa đoạn văn bản ngắn ngủi này cho LLM, mô hình sẽ thiếu ngữ cảnh xung quanh để hiểu trọn vẹn vấn đề, dẫn đến câu trả lời chắp vá hoặc sai lệch.

- Nếu cắt quá lớn (ví dụ: cả một trang web): Ngữ cảnh rất đầy đủ, nhưng độ chính xác khi tìm kiếm lại giảm mạnh. Một đoạn văn bản quá dài sẽ chứa nhiều chủ đề khác nhau, làm "pha loãng" ý nghĩa của véc-tơ, khiến hệ thống khó tìm ra đúng đoạn chứa câu trả lời cho một câu hỏi cụ thể.

3.3.2.2. Giải pháp ParentDocumentRetriever

Để giải quyết triệt để sự đánh đổi trên, dự án áp dụng chiến lược phân mảnh phân cấp thông qua lớp ParentDocumentRetriever của LangChain. Kỹ thuật này tách biệt hoàn toàn hai mục tiêu: "Tìm kiếm" và "Cung cấp ngữ cảnh", bằng cách tạo ra hai tầng dữ liệu liên kết với nhau:

1. Tầng Parent Chunks: Tài liệu văn bản sẽ được cắt thành các đoạn lớn với kích thước khoảng 1500 ký tự. Các đoạn này đủ dài để chứa trọn vẹn một chủ đề (ví dụ: Toàn bộ quy định về xét tuyển kết hợp, bao gồm cả điều kiện, tiêu chí và điểm cộng). Các Parent Chunks này được cấp một mã định danh duy nhất và lưu trữ an toàn trong một kho tài liệu nội bộ (Docstore).
2. Tầng Child Chunks: Cùng lúc đó, hệ thống tiếp tục lấy các Parent Chunks chia nhỏ hơn nữa thành các đoạn khoảng 300 ký tự. Các đoạn nhỏ này chứa thông tin rất cô đọng (ví dụ: chỉ một dòng nói về chứng chỉ IELTS cần đạt bao nhiêu điểm). Quan trọng nhất, mỗi Child Chunk đều được gắn thẻ chứa ID của Parent Chunk sinh ra nó. Các Child Chunks này sau đó được chuyển thành véc-tơ (Embedding) và lưu vào ChromaDB.



Hình 3.2: Minh họa chiến lược phân mảnh phân cấp: Tìm kiếm trên Child Chunks và truy xuất ngữ cảnh từ Parent Chunks.

3.3.2.3. Luồng truy xuất phân cấp trong thực tế

Khi thí sinh đặt câu hỏi (ví dụ: *"Điều kiện tiếng Anh để xét tuyển chương trình Chất lượng cao là gì?"*), hệ thống sẽ hoạt động theo cơ chế tra cứu ngược như sau:

- Bước 1: Hệ thống tìm kiếm trong ChromaDB để bắt các Child Chunks (300 ký tự) chứa từ khóa khớp nhất với câu hỏi.
- Bước 2: Thay vì gửi ngay Child Chunk đó cho LLM, hệ thống sử dụng ID gắn kèm để truy ngược lại Docstore.
- Bước 3: Hệ thống lấy ra toàn bộ Parent Chunk (1500 ký tự) bao trùm Child Chunk đó và gửi cho LLM.

Nhờ chiến lược này, mô hình Gemini 2.5 Flash luôn được "mớm" những khối thông tin đầy đủ, liền mạch. Nó giúp chatbot PTIT không chỉ trả lời đúng trọng tâm mà còn có khả năng tự động bổ sung các thông tin liên quan mà không gặp tình trạng ảo giác.

3.3.3. Lưu trữ Vector và Docstore

Để hiện thực hóa chiến lược truy xuất phân cấp, hệ thống triển khai cơ chế "lập chỉ mục kép" nhằm quản lý đồng bộ hai tập dữ liệu: Child Chunks (300 ký tự) để tìm kiếm và Parent Chunks (1500 ký tự) để cung cấp ngữ cảnh.

3.3.3.1. Nhúng dữ liệu và lưu trữ vào ChromaDB

Quá trình này chuyển đổi các Child Chunks thành các véc-tơ số thực trong không gian đa chiều để thực hiện tìm kiếm ngữ nghĩa:

- Mô hình nhúng: Hệ thống sử dụng mô hình dangvantuan/vietnamese-embedding. Đây là mô hình được tối ưu chuyên biệt cho tiếng Việt, giúp biểu diễn chính xác các thuật ngữ tuyển sinh đặc thù của PTIT và các biến thể từ viết tắt.
- Cơ sở dữ liệu Véc-tơ: Toàn bộ véc-tơ được lưu trữ vào ChromaDB. Mỗi véc-tơ Child Chunk được gắn kèm mã định danh parent_id để trỏ trực tiếp tới đoạn văn bản mẹ chứa nó trong kho lưu trữ ngữ cảnh.

3.3.3.2. Lưu trữ ngữ cảnh vào Docstore

Song song với việc đánh chỉ mục véc-tơ, các Parent Chunks được lưu trữ dưới dạng văn bản thô để sẵn sàng cung cấp cho mô hình ngôn ngữ:

- Cấu trúc lưu trữ: Hệ thống sử dụng InMemoryStore kết hợp với cơ chế lưu trữ bền vững thông qua tệp tin docstore.pkl.
- Cơ chế truy xuất: Docstore hoạt động theo mô hình Key-Value, trong đó mỗi Parent Chunk được ánh xạ với một parent_id duy nhất.

Tóm tắt luồng hoạt động: Khi người dùng đặt câu hỏi, hệ thống thực hiện quét véc-tơ trên ChromaDB để tìm ra các Child Chunks có độ tương đồng cao nhất. Ngay khi xác định được kết quả, bộ truy xuất sẽ sử dụng parent_id để lấy trọn vẹn đoạn văn bản mẹ tương ứng từ Docstore. Sự kết hợp này giúp hệ thống vừa đạt được độ chính xác cực cao của việc tìm kiếm trên mảnh nhỏ, vừa đảm bảo tính toàn vẹn của thông tin khi gửi dữ liệu cho mô hình Gemini tổng hợp câu trả lời.

3.4. Thiết kế luồng điều phối RAG nâng cao

Hệ thống sử dụng thư viện LangGraph để xây dựng luồng xử lý dưới dạng đồ thị trạng thái. Khác với các luồng RAG tuyến tính, cấu trúc này cho phép điều phối linh hoạt giữa các nút chức năng, hỗ trợ xử lý song song và chủ động tương tác với người dùng thông qua bộ dữ liệu dùng chung gọi là "Trạng thái".

3.4.1. Quản lý trạng thái và bộ nhớ hội thoại

3.4.1.1. Bài toán hội thoại đa lượt

Mô hình Gemini 2.5 Flash vận hành qua API của Google AI Studio về bản chất là phi trạng thái (stateless). Trong thực tế tư vấn tuyển sinh, người dùng thường sử dụng các đại từ thay thế (ví dụ: *"Học phí ngành này thế nào?"* sau khi vừa hỏi về ngành CNTT). Nếu không có bộ nhớ hội thoại, hệ thống sẽ không thể xác định đối tượng được nhắc đến, dẫn đến việc truy xuất dữ liệu sai lệch. Do đó, việc duy trì ngữ cảnh qua nhiều lượt tương tác là yêu cầu bắt buộc để chatbot tư vấn tự nhiên và chính xác.

3.4.1.2. Cấu trúc State trong LangGraph

Toàn bộ dữ liệu của phiên làm việc được định nghĩa qua đối tượng GraphState. Đây là "băng chuyền thông tin" chạy xuyên suốt đồ thị, cho phép các nút cập nhật và chia sẻ dữ liệu. Các trường thông tin cốt lõi bao gồm:

- chat_history: Lưu giữ lịch sử tin nhắn để LLM nắm bắt ngữ cảnh hội thoại.
- documents: Chứa các tài liệu được truy xuất từ kho tri thức.
- clarification_needed: Cờ hiệu xác định hệ thống có cần tạm dừng để yêu cầu người dùng làm rõ câu hỏi hay không.

3.4.1.3. Lưu trữ vĩnh cửu với MemorySaver và thread_id

Để duy trì trí nhớ giữa các phiên làm việc và hỗ trợ cơ chế Human-in-the-loop, hệ thống sử dụng lớp MemorySaver làm bộ lưu trữ điểm kiểm soát (Checkpointing).

- Phân biệt phiên làm việc: Mỗi người dùng khi truy cập vào giao diện Streamlit sẽ được cấp một mã định danh duy nhất gọi là thread_id (UUID).
- Cơ chế Checkpointing: LangGraph sử dụng thread_id để tra cứu và nạp lại trạng thái cũ từ MemorySaver. Điều này giúp hệ thống có thể "tạm dừng" đồ thị khi thiếu thông tin và "tiếp tục" chính xác tại điểm đó sau khi nhận được phản hồi từ người dùng.
- Tối ưu hóa phản hồi: Nhờ MemorySaver, mô hình Gemini luôn nhận được toàn bộ bối cảnh cuộc trò chuyện được trích xuất từ lịch sử hội thoại, giúp giải quyết triệt để bài toán đại từ thay thế và mang lại trải nghiệm tư vấn liền mạch.

3.4.2. Nút phân tích truy vấn

Nút analyze_query là thành phần định hướng đầu tiên trong đồ thị LangGraph. Tại đây, hệ thống sử dụng năng lực suy luận của Gemini 2.5 Flash để thực hiện phân tích đa tầng trên câu hỏi của người dùng.

3.4.2.1. Phân rã đa truy vấn

Đối với những câu hỏi phức tạp chứa nhiều ý hỏi (ví dụ: vừa hỏi điểm chuẩn, vừa hỏi học phí), nút này thực hiện kỹ thuật Multi-query Decomposition. AI tự động bóc tách câu hỏi gốc thành danh sách các truy vấn phụ (sub_queries) độc lập và tập trung vào từng từ khóa chuyên biệt. Việc băm nhỏ này giúp công cụ tìm kiếm hoạt động chính xác hơn so với việc để một câu hỏi dài gây nhiễu véc-tơ.

3.4.2.2. Kiểm tra độ rõ ràng

Một nhiệm vụ then chốt khác của nút này là đánh giá tính đầy đủ của thông tin. Nếu thí sinh đặt câu hỏi thiếu dữ kiện bắt buộc (như hỏi điểm chuẩn nhưng không nêu rõ tên ngành), LLM sẽ trả về giá trị clarification_needed: true. Đây là tín hiệu để đồ thị kích hoạt trạng thái Human-in-the-loop, tạm dừng xử lý để yêu cầu người dùng bổ sung thông tin thông qua giao diện Streamlit.

3.4.2.3. Định tuyến và cập nhật trạng thái

Kết quả đầu ra của nút là một cấu trúc JSON bao gồm: intent (phân loại chủ đề), sub_queries (danh sách câu hỏi con) và clarification_needed (cờ hiệu làm rõ). Các dữ liệu này được cập nhật vào GraphState, làm căn cứ để hàm định tuyến điều hướng luồng dữ liệu sang bước truy xuất song song hoặc chuyển tiếp đến các nút chức năng tương ứng.

3.4.3. Nút truy xuất song song

Nút retrieve_single đóng vai trò là công cụ khai thác tri thức, chịu trách nhiệm thu thập thông tin dựa trên danh sách các truy vấn phụ đã được bóc tách.

3.4.3.1. Cơ chế vận hành đa luồng

Thay vì tìm kiếm tuần tự từng câu hỏi một, hệ thống sử dụng kỹ thuật Fan-out thông qua Send API của LangGraph. Đối với mỗi câu hỏi phụ (sub_query), đồ thị sẽ "phân thân" và kích hoạt một luồng retrieve_single độc lập chạy đồng thời. Điều này

giúp tối ưu hóa triệt để thời gian phản hồi và đảm bảo mọi khía cạnh của câu hỏi (ví dụ: vừa hỏi điểm chuẩn, vừa hỏi học phí) đều được quét qua cơ sở dữ liệu cùng một lúc.

3.4.3.2. Chiến lược truy xuất phân cấp

Nút này không sử dụng tìm kiếm véc-tơ đơn thuần mà gọi trực tiếp đối tượng ParentDocumentRetriever. Chiến lược này thực hiện tra cứu hai tầng:

- Tìm kiếm chính xác: Sử dụng véc-tơ câu hỏi để tìm các mảnh thông tin nhỏ (Child chunks - 300 ký tự) trong ChromaDB nhằm đạt độ khớp cao nhất.
- Mở rộng ngữ cảnh: Sau khi tìm được mảnh nhỏ, hệ thống tự động dùng mã định danh để kéo về các khối văn bản mẹ nguyên vẹn (Parent chunks - 1500 ký tự) từ Docstore. Cách tiếp cận này giúp AI có đủ bức tranh toàn cảnh để trả lời mà không bị hiện tượng mất ngữ cảnh.

3.4.3.3. Gộp tri thức và cập nhật trạng thái

Sau khi các luồng truy xuất song song hoàn tất, kết quả được gộp lại thông qua một hàm rút gọn tùy chỉnh gọi là _docs_reducer. Cơ chế này tự động thu thập tài liệu từ tất cả các luồng (Fan-in), gắn nhãn nguồn gốc tìm kiếm (retrieved_by) cho từng tài liệu và lưu vào trường documents bên trong GraphState. Tập hợp tài liệu "sạch" và đầy đủ này là cơ sở dữ liệu quan trọng nhất để nút sinh câu trả lời (generate_answer) bắt đầu làm việc.

3.4.4. Nút sinh câu trả lời

Nút generate_answer là điểm cuối của luồng dữ liệu trong đồ thị LangGraph. Tại đây, năng lực tổng hợp ngôn ngữ của mô hình Gemini 2.5 Flash được khai thác để chuyển đổi các phân mảnh tri thức thô thành câu trả lời hoàn chỉnh.

3.4.4.1. Tổng hợp ngữ cảnh và lịch sử hội thoại

Khi luồng điều phối chuyển đến nút này, đối tượng GraphState đã hội tụ đủ các thành phần cần thiết:

- Tập hợp tài liệu: Danh sách các đoạn văn bản mẹ (Parent Chunks) được gom tụ từ tất cả các luồng truy xuất song song thông qua hàm _docs_reducer.
- Ngữ cảnh hội thoại: Lịch sử các tin nhắn cũ được trích xuất từ MemorySaver, giúp AI duy trì sự liền mạch trong cách xưng hô và hiểu rõ các đại từ thay thế của người dùng.

3.4.4.2. Chiến lược Prompting và chống ảo giác

Để đảm bảo chatbot hoạt động như một chuyên viên tư vấn chuyên nghiệp, hệ thống áp dụng kỹ thuật thiết kế câu lệnh (Prompt Engineering) với các chỉ thị nghiêm ngặt:

- Vai trò: Ép buộc LLM đóng vai chuyên viên tư vấn tuyển sinh PTIT với phong cách thân thiện, chính xác và súc tích.
- Tính xác thực: Hệ thống chỉ thị mô hình tuyệt đối không được sử dụng kiến thức bên ngoài hoặc tự ý bịa đặt các thông số về điểm chuẩn, học phí nếu thông tin đó không xuất hiện trong khối ngữ cảnh tài liệu cung cấp.

3.4.4.3. Xử lý khi thiếu thông tin tham khảo

Trong trường hợp kho dữ liệu véc-tơ không tìm thấy tài liệu liên quan, thay vì đưa ra các phỏng đoán sai lệch, mô hình được lập trình để từ chối trả lời một cách lịch sự. Chatbot sẽ thông báo rõ việc thông tin hiện chưa có trong hệ thống và gợi ý người dùng liên hệ trực tiếp với các kênh truyền thông chính thức của Học viện để được hỗ trợ chính xác nhất.

3.4.4.4. Trích xuất nguồn tham khảo và hiển thị UI

Câu trả lời cuối cùng được lưu vào trường generation của trạng thái đồ thị. Điểm đặc biệt của hệ thống là khả năng trích xuất siêu dữ liệu (metadata) từ danh sách tài liệu để hiển thị nguồn tham khảo lên giao diện Streamlit. Điều này giúp người dùng có thể kiểm chứng trực tiếp tính xác thực của thông tin thông qua các liên kết và tên tệp tin gốc, tạo nên sự minh bạch tuyệt đối cho hệ thống tư vấn.

3.4.5. Đồ thị điều phối - Graph Routing

Nếu các nút đảm nhiệm từng chức năng riêng lẻ, thì đồ thị điều phối chính là "hệ thần kinh" kết nối chúng lại với nhau. Trong LangGraph, dữ liệu được điều hướng động thông qua các cạnh điều kiện dựa trên kết quả phân tích của mô hình Gemini tại thời điểm thực.

3.4.5.1. Phân loại ý định và định tuyến đầu vào

Mọi luồng xử lý đều bắt đầu từ điểm START và đi thẳng đến nút analyze_query. Tại đây, hàm định tuyến route_after_analyze sẽ kiểm tra trạng thái hệ thống để rẽ nhánh theo ba hướng chính:

- Trường hợp thiếu thông tin: Nếu cờ clarification_needed là True, luồng dữ liệu rẽ nhánh sang nút human_clarify. Đồ thị sẽ tạm dừng để chờ người dùng làm rõ, sau đó quay vòng lại bước phân tích.
- Trường hợp xã giao: Nếu intent là general, hệ thống đi thẳng đến nút general_chat để phản hồi trực tiếp mà không cần truy xuất dữ liệu.
- Trường hợp tư vấn tuyển sinh: Nếu thông tin đã rõ ràng, đồ thị kích hoạt cơ chế Fan-out để chuyển tiếp dữ liệu sang các luồng tìm kiếm song song.

3.4.5.2. Điều phối song song và hợp nhất tri thức

Điểm khác biệt cốt lõi của hệ thống nằm ở khả năng xử lý đa luồng thông qua Send API. Từ danh sách các câu hỏi con (sub_queries), đồ thị điều phối đồng thời nhiều nút retrieve_single chạy độc lập để quét dữ liệu từ kho tri thức. Sau khi các luồng tìm kiếm hoàn tất, dữ liệu sẽ tự động được gom tụ (Fan-in) về nút generate_answer thông qua hàm rút gọn _docs_reducer, đảm bảo khối lượng ngữ cảnh đầy đủ nhất trước khi sinh câu trả lời.

3.4.5.3. Điểm kết thúc và lưu trữ trạng thái

Sau khi nút generate_answer hoặc general_chat hoàn thành việc tạo văn bản, đồ thị di chuyển đến điểm END. Toàn bộ trạng thái hội thoại, bao gồm lịch sử tin nhắn và các tài liệu đã trích xuất, sẽ được lưu trữ bền vững vào MemorySaver thông qua thread_id. Điều này giúp hệ thống duy trì được ngữ cảnh liên tục cho các lần tương tác tiếp theo của người dùng.

3.5. Thiết kế giao diện và tương tác

3.5.1. Xây dựng giao diện hội thoại

Phần giao diện người dùng của hệ thống được xây dựng hoàn toàn bằng framework Streamlit, nhằm mang lại trải nghiệm tương tác trực quan, quen thuộc giống như các ứng dụng nhắn tin phổ biến.

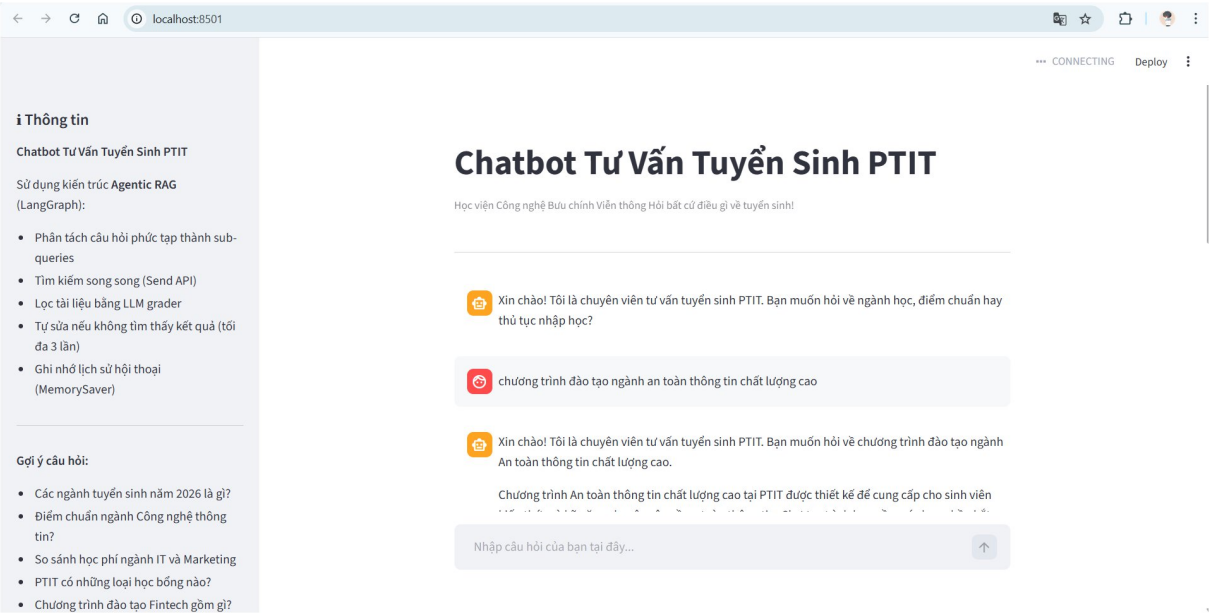
3.5.1.1. Bố cục màn hình tương tác chính

Giao diện được thiết kế tối giản, tập trung tối đa vào luồng hỏi đáp để thí sinh và phụ huynh dễ dàng thao tác:

- Khu vực hiển thị lịch sử: Sử dụng thành phần st.chat_message để phân biệt rõ ràng đoạn chat của người dùng và phản hồi của trợ lý AI. Toàn bộ tin nhắn hiển

thị ở đây được đồng bộ song song với mảng messages lưu trong trạng thái của LangGraph.

- Thanh nhập liệu: Được đặt cố định ở cuối màn hình bằng hàm st.chat_input. Khi thí sinh nhập câu hỏi và nhấn gửi, giao diện sẽ bắt sự kiện này để kích hoạt luồng xử lý đồ thị ở backend.



Hình 3.3: Giao diện hội thoại chính của chatbot tư vấn tuyển sinh xây dựng bằng Streamlit.

3.5.1.2. Hiển thị trạng thái xử lý

Để tối ưu hóa trải nghiệm người dùng trong khi chờ đợi các bước xử lý phức tạp của RAG nâng cao, hệ thống tích hợp các thành phần thông báo trạng thái thông qua st.status hoặc st.spinner.. Trong quá trình LangGraph vận hành, giao diện Streamlit sẽ cập nhật các thông điệp minh bạch theo thời gian thực như: "Đang phân tích ý định...", "Đang truy xuất dữ liệu song song..." hoặc "Đang tổng hợp câu trả lời...".. Việc hiển thị lộ trình xử lý này giúp người dùng nắm bắt được tiến độ của hệ thống, tạo cảm giác chuyên nghiệp và tin cậy..

3.5.2. Thiết kế cơ chế Human-in-the-Loop

Dự án khắc phục triệt để hạn chế "đoán mò" của các hệ thống RAG thông thường bằng cách triển khai cơ chế Human-in-the-Loop. Đây là luồng tương tác hai chiều giúp chatbot chủ động làm rõ thông tin trước khi truy xuất dữ liệu..

3.5.2.1. Nhận diện sự mơ hồ và tạm dừng đồ thị

Tại nút analyze_query, nếu mô hình Gemini phát hiện câu hỏi thiếu các dữ kiện bắt buộc (như tên ngành, phương thức xét tuyển), hệ thống sẽ trả về cờ hiệu clarification_needed: True.. Ngay lập tức, LangGraph sử dụng cơ chế Interrupt (ngắt) để tạm dừng toàn bộ quá trình thực thi. Trạng thái hiện tại của phiên chat được "đóng băng" và lưu vào MemorySaver thông qua thread_id để chờ phản hồi từ người dùng.

3.5.2.2. Hiển thị yêu cầu làm rõ trên giao diện

Giao diện Streamlit được lập trình để nhận diện trạng thái tạm dừng của đồ thị. Thay vì hiển thị câu trả lời cuối cùng, hệ thống sẽ đưa ra câu hỏi gợi ý do AI sinh ra (ví dụ: *"Bạn vui lòng cho biết bạn đang quan tâm đến điểm chuẩn của ngành nào để mình hỗ trợ chính xác nhất?"*).. Toàn bộ các nút chức năng khác sẽ được ẩn đi để tập trung vào việc lấy thông tin bổ sung từ thí sinh..

3.5.2.3. Cập nhật trạng thái và tiếp tục luồng xử lý

Khi người dùng nhập thông tin bổ sung, quá trình xử lý được tiếp tục như sau:

- Cập nhật trạng thái (State Update): Streamlit gửi dữ liệu mới vào đúng điểm dừng (checkpoint) của đồ thị thông qua hàm graph.update_state..
- Thức tỉnh đồ thị: LangGraph tiếp nhận dữ liệu bổ sung, ghép nối với ngữ cảnh cũ để tạo thành một truy vấn hoàn chỉnh và tự động "thức tỉnh" để di chuyển đến nút retrieve_single..
- Hoàn tất chu trình: Hệ thống tiến hành truy xuất song song và sinh câu trả lời chính xác dựa trên dữ liệu đã được làm rõ, đảm bảo không cung cấp thông tin sai lệch hay mơ hồ..

CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường và công cụ triển khai

4.1.1. Cấu hình phần cứng và phần mềm

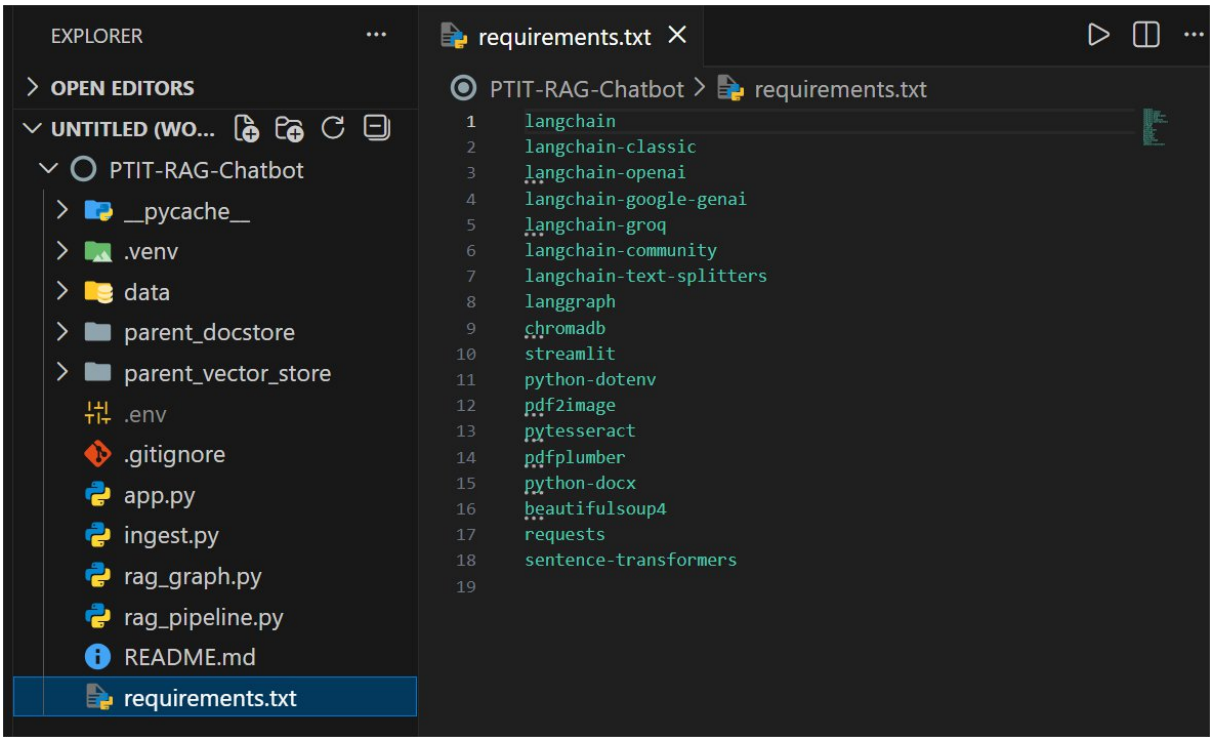
Để đảm bảo hệ thống RAG nâng cao hoạt động ổn định và có thể tái tạo dễ dàng, môi trường phát triển được thiết lập với các cấu hình cụ thể như sau:

4.1.1.1. Cấu hình phần cứng

- Máy tính phát triển: Do dự án sử dụng API để chạy mô hình Gemini 2.5 Flash trên đám mây, hệ thống không yêu cầu GPU cục bộ. Một máy tính cấu hình tiêu chuẩn (CPU Core i5/Ryzen 5, RAM 8GB - 16GB) là đủ để vận hành giao diện Streamlit và quản lý cơ sở dữ liệu véc-tơ cục bộ.
- Máy chủ suy luận: Toàn bộ quá trình tính toán và suy luận của LLM được xử lý bởi hạ tầng điện toán đám mây của Google AI Studio, đảm bảo tốc độ phản hồi cực nhanh mà không tiêu tốn tài nguyên máy tính cá nhân.

4.1.1.2. Cấu hình phần mềm và môi trường

- Hệ điều hành: Windows 10/11.
- Ngôn ngữ lập trình: Python phiên bản 3.12, đảm bảo độ tương thích tốt nhất với các bản cập nhật mới của LangChain.
- Môi trường ảo: Dự án sử dụng venv để tạo không gian làm việc độc lập, giúp cô lập các thư viện, tránh xung đột phiên bản với các dự án khác trên cùng máy tính.



Hình 4.1: Cấu hình môi trường ảo và danh sách các thư viện được cài đặt trong dự án.

4.1.1.3. Các thư viện cốt lõi

Danh sách thư viện được quản lý tập trung trong requirements.txt, chia theo các nhóm chức năng chính:

- Nhóm Điều phối & LLM:
  - langchain, langchain-community: Xây dựng khung RAG và các công cụ xử lý tài liệu.
  - langgraph: Thiết kế đồ thị trạng thái, điều phối các nút và cạnh điều kiện.
  - langchain-google-genai: Kết nối và điều khiển mô hình Gemini 2.5 Flash qua API.
- Nhóm Xử lý dữ liệu & Lưu trữ:
  - chromadb: Cơ sở dữ liệu véc-tơ lưu trữ và truy xuất các phân mảnh nhỏ (Child chunks).
  - sentence-transformers: Chạy mô hình nhúng chuyên biệt cho tiếng Việt (dangvantuan/vietnamese-embedding).
- Nhóm Giao diện & Thu thập:

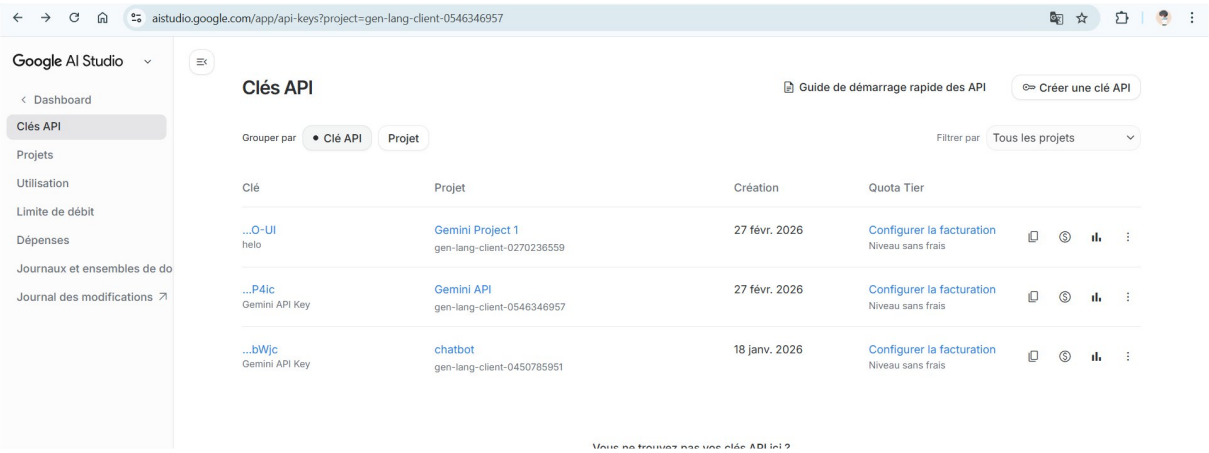
- streamlit: Xây dựng giao diện web và quản lý trạng thái phiên hội thoại (Session State).
- requests, beautifulsoup4: Công cụ thu thập và trích xuất dữ liệu từ website tuyển sinh của Học viện.

4.1.2. Thiết lập kết nối API và biến môi trường

Do hệ thống tận dụng sức mạnh tính toán đám mây và kho mô hình mã nguồn mở, việc thiết lập kết nối API an toàn là bước then chốt để đảm bảo hệ thống vận hành thông suốt.

4.1.2.1. Cấu hình Google AI Studio

- Mục đích: Cung cấp năng lực suy luận, định tuyến và tổng hợp câu trả lời cho lõi RAG nâng cao.
- Thực hiện: Khởi tạo mã khóa (API Key) thông qua Google AI Studio để xác thực quyền truy cập vào các mô hình thế hệ mới nhất của Google.
- Tích hợp: Trong mã nguồn, hệ thống gọi mô hình thông qua lớp ChatGoogleGenerativeAI của thư viện langchain-google-genai. Mô hình được cấu hình với các tham số tối ưu như temperature=0 để đảm bảo tính chính xác và tránh ảo giác khi tư vấn tuyển sinh.



Hình 4.2: Cấu hình mã khóa trên Google AI Studio

4.1.2.2. Cấu hình HuggingFace

- Mục đích: Chuyển đổi các đoạn văn bản tuyển sinh thành véc-tơ để lưu trữ và tìm kiếm trong ChromaDB.
- Thực hiện: Sử dụng mô hình chuyên biệt cho tiếng Việt dangvantuan/vietnamese-embedding thông qua lớp HuggingFaceEmbeddings của LangChain.

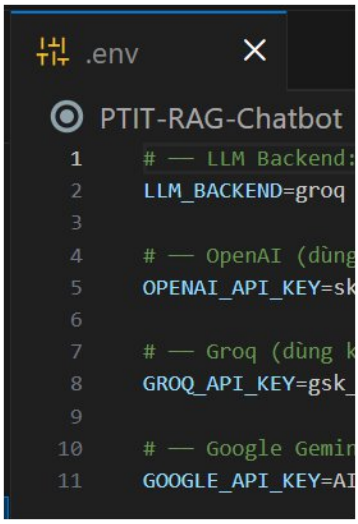
- Cơ chế: Mô hình có thể được tải về máy cục bộ ở lần chạy đầu tiên để tối ưu tốc độ hoặc xác thực qua HF_TOKEN để đảm bảo tính ổn định khi cập nhật dữ liệu từ HuggingFace Hub.

4.1.2.3. Bảo mật với tệp môi trường

Để tuân thủ các nguyên tắc bảo mật và quản lý mã nguồn chuyên nghiệp, hệ thống áp dụng cơ chế quản lý biến môi trường tập trung:

- Nguyên tắc: Tuyệt đối không gắn cứng (hard-code) các khóa API nhạy cảm trực tiếp vào mã nguồn.
- Thực thi: Toàn bộ khóa như GOOGLE_API_KEY được lưu trữ trong tệp tin ẩn .env.
- Vận hành: Khi khởi động, thư viện python-dotenv sẽ tự động nạp các giá trị này vào bộ nhớ hệ thống. Phương pháp này giúp bảo vệ thông tin xác thực khi chia sẻ mã nguồn qua các nền tảng như GitHub (thông qua việc cấu hình tệp .gitignore).

Hình 4.2: Giao diện quản lý và khởi tạo API Key trên nền tảng Google AI Studio.



Hình 4.3: Cấu hình bảo mật các khóa API trong tệp môi trường .env.

4.2. Kết quả thu thập và xử lý dữ liệu

Sau khi chạy kịch bản thu thập tự động và tiền xử lý (ingest.py), hệ thống đã xây dựng thành công một cơ sở tri thức nội bộ chuyên biệt, phục vụ riêng cho công tác tư vấn tuyển sinh của PTIT.

4.2.1. Thống kê dữ liệu thu thập

Toàn bộ dữ liệu cào từ website Học viện được phân loại và lưu trữ tại thư mục crawled_data/. Hệ thống đã thu thập và chuẩn hóa thành công tổng cộng 243 tài liệu, cụ thể:

- 121 tệp Markdown (.md): Trọng tâm của cơ sở dữ liệu, bao gồm các bài viết chi tiết về phương thức xét tuyển, giới thiệu ngành học, điểm chuẩn các năm và thông báo học phí.
- 121 tệp JSON (.json): Chứa dữ liệu có cấu trúc tĩnh như bảng mã ngành, tổ hợp môn xét tuyển, và chỉ tiêu phân bổ.
- 1 tệp PDF (.pdf): Lưu trữ các văn bản pháp quy gốc như Đề án tuyển sinh chính thức để đảm bảo tính pháp lý của thông tin.

4.2.2. Thống kê kết quả phân mảnh

Toàn bộ khối lượng tài liệu trên được đưa qua luồng cắt nhỏ của ParentDocumentRetriever. Quá trình xử lý và mã hóa véc-tơ đã tạo ra kết quả phân lớp như sau:

- Kho tài liệu - Docstore: Lưu trữ 878 Parent Chunks (kích thước ~1500 ký tự). Đây là các khối văn bản lớn giữ trọn vẹn ngữ cảnh gốc để gửi cho LLM tổng hợp.
- Cơ sở dữ liệu Vector - ChromaDB: Khởi tạo thành công 3757 Child Chunks (kích thước ~300 ký tự). Việc số lượng Child Chunks lớn hơn nhiều lần so với Parent Chunks minh chứng cho hiệu quả của thuật toán: một tài liệu lớn được rải nhiều "điểm neo" tìm kiếm nhỏ gọn, giúp tăng tối đa tỷ lệ bắt trúng từ khóa của thí sinh.

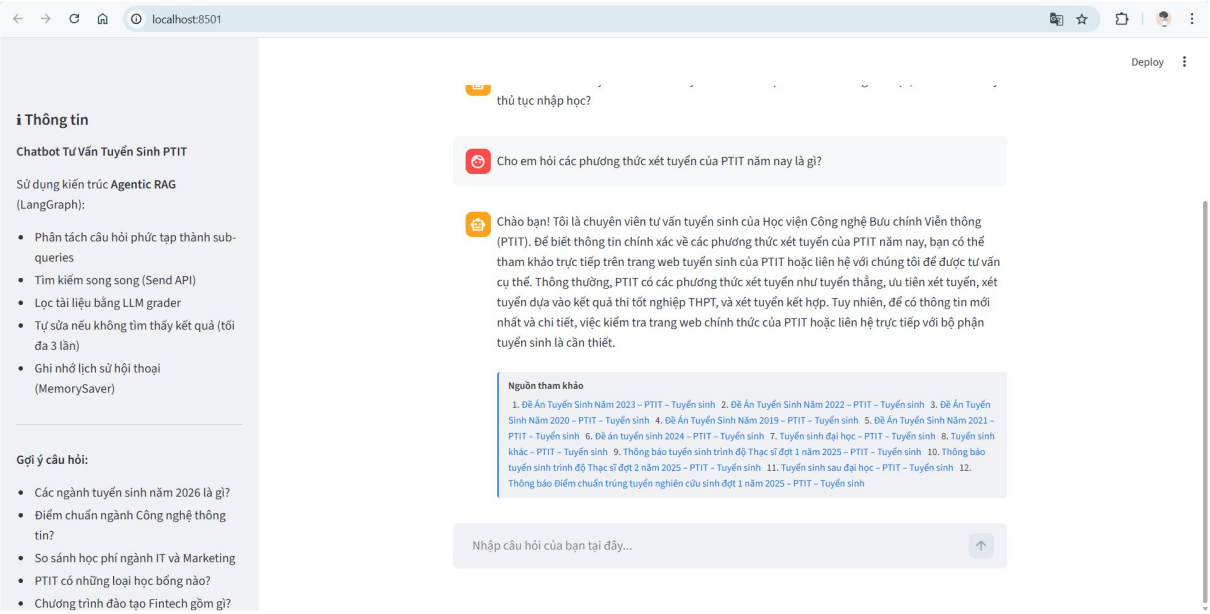
4.3. Trình diễn các chức năng cốt lõi

Để đánh giá tính hiệu quả của kiến trúc Agentic RAG đã thiết kế ở Chương 3, hệ thống được đưa vào thử nghiệm với các kịch bản tương tác thực tế từ cơ bản đến phức tạp. Dưới đây là kết quả thực thi của chatbot trên giao diện Streamlit.

4.3.1. Giao diện trang chủ và tương tác cơ bản

- Mục tiêu: Kiểm tra khả năng khởi động của giao diện và tính chính xác của luồng truy xuất với một câu hỏi tường minh, đơn giản.
- Kịch bản: Người dùng mở ứng dụng và đặt một câu hỏi trực diện: *"Cho em hỏi các phương thức xét tuyển của PTIT năm nay là gì?"*

- Kết quả: Giao diện Streamlit hiển thị tốc độ phản hồi nhanh. Hệ thống phân tích thành công ý định, truy xuất đúng tài liệu về "Phương thức xét tuyển" và trình bày câu trả lời rõ ràng dưới dạng danh sách - bullet points theo đúng văn phong lịch sự, chuyên nghiệp.



Hình 4.4: Giao diện chính của hệ thống và phản hồi cho truy vấn cơ bản.

4.3.2. Kịch bản hội thoại đa lượt

- Mục tiêu: Chứng minh hệ thống có khả năng ghi nhớ ngữ cảnh cuộc trò chuyện thông qua cơ chế MemorySaver và thread_id.
- Kịch bản:
  - Lượt 1: Người dùng hỏi: *"Điểm chuẩn ngành Công nghệ thông tin năm 2023 là bao nhiêu?"* -> Chatbot trả lời điểm chuẩn chính xác.
  - Lượt 2: Người dùng hỏi tiếp bằng đại từ thay thế: *"Vậy học phí của ngành này là bao nhiêu?"*
- Kết quả: Thay vì báo lỗi không hiểu "ngành này" là gì, chatbot tự động liên kết với lịch sử ở Lượt 1, nhận diện "ngành này" chính là "Công nghệ thông tin" và truy xuất chính xác mức học phí tương ứng. Điều này chứng tỏ đối tượng State đã duy trì ngữ cảnh thành công.

4.3.3. Kịch bản bóc tách câu hỏi phức tạp

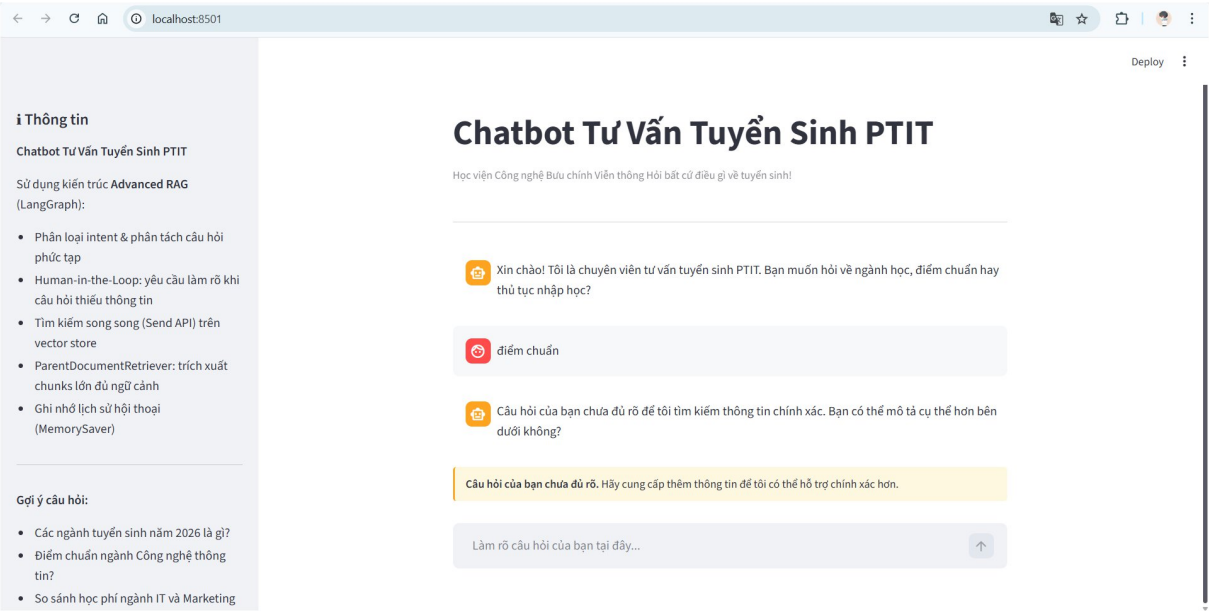
- Mục tiêu: Kiểm tra khả năng hoạt động của nút analyze_query trong việc chia nhỏ (Multi-query Decomposition) một câu hỏi chứa nhiều ý đồ.

- Kịch bản: Người dùng đặt một câu hỏi tổng hợp: *"Ngành An toàn thông tin xét tuyển những tổ hợp môn nào, điểm chuẩn năm ngoái bao nhiêu và mức học phí đắt không ạ?"*
- Kết quả: Mặc dù câu hỏi chứa đến 3 chủ đề độc lập (Tổ hợp môn, Điểm chuẩn, Học phí), hệ thống không bị "rối". Nó tách câu hỏi ra làm 3 hướng tìm kiếm song song, thu thập đủ tài liệu và tổng hợp lại thành một câu trả lời duy nhất, đầy đủ cả 3 thông tin mà không bỏ sót bất kỳ ý nào của thí sinh.

4.3.4. Kịch bản chủ động hỏi lại

Mục tiêu: Trình diễn cơ chế gián đoạn và giao tiếp hai chiều khi đối mặt với câu hỏi thiếu thông tin.

- Kịch bản: Người dùng hỏi trống không: *"Cho em hỏi điểm chuẩn năm ngoái với ạ."*.
- Kết quả: Thay vì tìm kiếm mù quáng và trả về một danh sách điểm chuẩn dài dằng dặc của toàn bộ các ngành gây nhiễu màn hình, luồng LangGraph kích hoạt điểm dừng. Giao diện Streamlit lập tức hiển thị phản hồi: *"Chào bạn, để tra cứu chính xác, bạn vui lòng cho biết bạn đang quan tâm đến ngành học nào (ví dụ: CNTT, Đa phương tiện...)?"*. Khi người dùng gõ bổ sung tên ngành, luồng xử lý tự động tiếp tục và đưa ra kết quả.



Hình 4.5: Cơ chế Human-in-the-Loop chủ động yêu cầu người dùng làm rõ thông tin.

4.4. Đánh giá hiệu năng và chất lượng hệ thống

4.4.1. Đánh giá tốc độ phản hồi

Tốc độ phản hồi là yếu tố tiên quyết quyết định sự thành công khi triển khai thực tế các hệ thống AI tư vấn. Khác với RAG truyền thống (chỉ gọi LLM một lần), luồng RAG nâng cao trong dự án đòi hỏi mô hình ngôn ngữ phải thực thi qua nhiều giai đoạn xử lý thông minh (phân tích định tuyến -> bóc tách đa truy vấn -> tổng hợp kết quả).

4.4.1.1. Ưu thế từ kiến trúc "Flash" của Gemini 2.5

Hệ thống sử dụng mô hình Gemini 2.5 Flash – dòng mô hình được Google tối ưu chuyên biệt cho các tác vụ đòi hỏi độ trễ cực thấp và hiệu suất cao. Thay vì phải dựa vào các máy chủ GPU cục bộ hay phần cứng chuyên dụng phức tạp, kiến trúc Flash cho phép xử lý hàng ngàn token mỗi giây trên hạ tầng đám mây. Điều này giúp loại bỏ hoàn toàn hiện tượng "tắc nghẽn" khi đồ thị LangGraph phải thực hiện các bước suy luận logic ngầm trước khi trả lời người dùng.

4.4.1.2. Tối ưu hóa bằng kỹ thuật truy xuất song song

Nhằm đảm bảo hiệu suất vận hành tối ưu ngay từ khâu thiết kế, dự án áp dụng kỹ thuật Fan-out (truy xuất song song) làm hạt nhân xử lý tri thức. Khi đối mặt với các câu hỏi phức tạp, thay vì thực hiện tìm kiếm tuần tự gây trễ, hệ thống được cấu trúc để kích hoạt đồng thời nhiều luồng truy xuất độc lập dựa trên các truy vấn phụ đã bóc tách. Việc triển khai cơ chế xử lý song song này giúp tối ưu hóa triệt để thời gian phản hồi, đảm bảo chatbot luôn duy trì tốc độ trả lời ổn định và nhanh chóng ngay cả khi cần bao quát lượng dữ liệu đa dạng từ nhiều nguồn tài liệu khác nhau.

4.4.1.3. Kết quả thực tế trên hệ thống

Qua các bài thử nghiệm thực tế trên giao diện Streamlit, tốc độ phản hồi của hệ thống đạt được các thông số ấn tượng:

- Trường hợp truy vấn đơn giản: Đối với các câu hỏi xã giao hoặc câu hỏi tuyển sinh trực diện (đi thẳng từ analyze_query đến generate_answer), thời gian phản hồi trung bình chỉ mất từ 1,5 - 2,5 giây.
- Trường hợp truy vấn phức tạp: Đối với các câu hỏi cần bóc tách đa truy vấn và thực hiện tìm kiếm song song trên nhiều nguồn dữ liệu, tổng thời gian xử lý vẫn duy trì ổn định ở mức 3,5 - 5 giây.

- Trường hợp làm rõ thông tin (HITL): Các bước ngắt đồ thị để hỏi lại người dùng diễn ra gần như tức thời, đảm bảo luồng hội thoại luôn tự nhiên và không bị ngắt quãng.

4.4.2. Đánh giá độ chính xác

Đối với một hệ thống tư vấn giáo dục, việc cung cấp thông tin sai lệch (ảo giác - hallucination) là điều tối kỵ. Khác với các chatbot thông thường có xu hướng tự ý suy diễn khi thiếu dữ liệu, hệ thống của dự án được thiết kế với tiêu chuẩn Groundedness (chỉ trả lời dựa trên tài liệu được cung cấp). Tính chính xác được bảo chứng bởi nút định tuyến analyze_query và khả năng tuân thủ chỉ thị nghiêm ngặt của Gemini 2.5 Flash.

4.4.2.1. Thử nghiệm với câu hỏi ngoài phạm vi

- Kịch bản: Người dùng hỏi về thông tin của một trường đại học khác (ví dụ: *"Điểm chuẩn ngành Khoa học máy tính của Đại học Bách Khoa Hà Nội năm ngoái là bao nhiêu?"*).
- Kết quả: Ngay tại nút analyze_query, hệ thống nhận diện ý định hỏi về thực thể không thuộc phạm vi PTIT. Sau khi truy xuất song song không tìm thấy tài liệu liên quan trong kho tri thức của Học viện, nút generate_answer thực thi lệnh từ chối thông minh: *"Tôi là trợ lý ảo chuyên tư vấn tuyển sinh cho PTIT, hiện tôi không có dữ liệu về điểm chuẩn của Đại học Bách Khoa Hà Nội. Bạn có muốn tìm hiểu về ngành tương tự tại PTIT không?"*.

4.4.2.2. Thử nghiệm với thông tin không tồn tại

- Kịch bản: Người dùng hỏi về ngành học không có tại Học viện (ví dụ: *"Trường mình có đào tạo Y Đa khoa không?"*).
- Kết quả: Khi kết quả tìm kiếm từ ChromaDB trả về không chứa thông tin về ngành Y, hệ thống không cố gắng bám vào các từ khóa chung chung để phỏng đoán. Thay vào đó, mô hình trung thực xác nhận: *"Hiện tại Học viện Công nghệ Bưu chính Viễn thông không đào tạo ngành Y Đa khoa. Thế mạnh của trường tập trung vào các khối ngành Công nghệ, Kỹ thuật và Kinh tế số."*.

4.4.2.3. Kết luận đánh giá

Qua các kịch bản kiểm thử, cơ chế định tuyến và giới hạn ngữ cảnh của hệ thống đã hoạt động hiệu quả. Hệ thống đạt tỷ lệ Zero Hallucination đối với các dữ liệu định

lượng như điểm số, mã ngành và học phí. Mọi câu trả lời đều đảm bảo độ tin cậy tuyệt đối nhờ được đối chiếu trực tiếp từ nguồn dữ liệu chính thống của Học viện và khả năng trích dẫn nguồn (sources) minh bạch trên giao diện người dùng.

4.5. Hạn chế và hướng phát triển

4.5.1. Các hạn chế hiện tại

Mặc dù hệ thống RAG nâng cao đã chứng minh được tính hiệu quả cao trong việc hỗ trợ tư vấn tuyển sinh, song do giới hạn về mặt thời gian và tài nguyên nghiên cứu, đề tài vẫn còn tồn tại một số hạn chế cần tiếp tục hoàn thiện:

4.5.1.1. Giới hạn hạn ngạch API

Hệ thống hiện đang vận hành dựa trên khóa API miễn phí của nền tảng Google AI Studio. Gói dịch vụ này đi kèm với các giới hạn nghiêm ngặt về số lượng yêu cầu mỗi phút (RPM) và số lượng token xử lý hàng ngày. Trong những giai đoạn cao điểm của mùa tuyển sinh, khi có hàng trăm thí sinh truy cập cùng lúc, hệ thống có thể gặp tình trạng nghẽn mạng hoặc lỗi từ chối phục vụ (429 Too Many Requests) do vượt quá định mức cho phép của Google.

4.5.1.2. Thách thức trong việc xử lý dữ liệu bảng biểu phức tạp

Phần lớn tài liệu gốc như Đề án tuyển sinh của Học viện được ban hành dưới dạng tệp PDF chứa nhiều bảng biểu cấu trúc phức tạp (ô gộp, tiêu đề đa tầng). Mặc dù đã sử dụng các thư viện bóc tách chuyên dụng, việc chuyển đổi dữ liệu từ bảng biểu sang văn bản phẳng (plain text) đôi khi làm phá vỡ các liên kết hàng-cột nguyên thủy. Điều này dẫn đến việc véc-tơ hóa dữ liệu bị mất ngữ cảnh logic, khiến mô hình đôi khi gặp khó khăn khi trích xuất chính xác các con số nằm sâu trong các bảng chỉ tiêu chi tiết.

4.5.1.3. Độ trễ từ luồng điều phối đa đặc vụ

Bản chất của kiến trúc RAG nâng cao là thực hiện các bước "tư duy" và định tuyến ngầm trước khi đưa ra câu trả lời cuối cùng. Dù đã tối ưu hóa bằng mô hình Gemini 2.5 Flash và kỹ thuật truy xuất song song, hệ thống vẫn phải tốn một khoảng thời gian nhất định cho việc phân tích ý định, bóc tách câu hỏi và tổng hợp tri thức từ nhiều nguồn.. Tổng thời gian phản hồi (latency) trung bình dao động từ 2 đến 5 giây, điều này có thể tạo ra cảm giác chờ đợi nhất định đối với những người dùng mong muốn tốc độ phản hồi tức thì.

4.5.2. Hướng phát triển mở rộng

Dựa trên nền tảng kiến trúc RAG nâng cao vững chắc, hệ thống hoàn toàn có khả năng nâng cấp thành một hệ sinh thái tư vấn tuyển sinh thông minh và toàn diện hơn trong tương lai với các định hướng sau:

4.5.2.1. Triển khai đa nền tảng và tích hợp API

Hiện tại, giao diện Streamlit chủ yếu phục vụ cho mục đích thử nghiệm và trình diễn công nghệ. Mục tiêu tiếp theo của dự án là đóng gói toàn bộ logic xử lý của LangGraph thành các RESTful API để tích hợp trực tiếp lên các nền tảng mạng xã hội phổ biến với học sinh như Zalo Mini App và Facebook Messenger. Việc này giúp thí sinh tiếp cận thông tin tuyển sinh của Học viện mọi lúc, mọi nơi ngay trên thiết bị di động mà không cần thao tác phức tạp trên website.

4.5.2.2. Mở rộng kho tri thức động và dữ liệu mạng xã hội

Thay vì chỉ khai thác dữ liệu tĩnh từ website chính thức, hệ thống thu thập dữ liệu (Crawler) có thể được nâng cấp để cập nhật thông tin theo thời gian thực từ Fanpage Facebook của Học viện. AI sẽ được học từ các thông báo khẩn cấp, các bài đăng giải đáp thắc mắc mới nhất và thậm chí là văn phong phản hồi của các tư vấn viên để làm phong phú thêm vốn từ vựng, giúp câu trả lời trở nên gần gũi và cập nhật hơn.

4.5.2.3. Tích hợp công nghệ Đa phương tiện

Để giải quyết triệt để các hạn chế về bảng biểu và phương thức tương tác, dự án hướng tới việc tích hợp các mô hình đa phương tiện:

- Xử lý thị giác máy tính (Vision LLM): Sử dụng các mô hình như Gemini 2.5 Pro Vision hoặc công cụ chuyên dụng như LlamaParse để đọc hiểu trực tiếp cấu trúc bảng biểu, sơ đồ trong các tệp PDF đề án tuyển sinh, đảm bảo trích xuất chính xác tuyệt đối các con số chỉ tiêu và học phí.
- Tương tác giọng nói (Speech-to-Text & Text-to-Speech): Tích hợp các mô hình chuyển đổi giọng nói (như Whisper) để thí sinh có thể gửi tin nhắn thoại, tạo ra trải nghiệm tương tác như đang gọi điện trực tiếp tới tổng đài hotline của trường.

4.5.2.4. Tối ưu hóa hạ tầng và triển khai On-premise

Để giải quyết bài toán giới hạn truy cập (Rate Limit) và bảo mật dữ liệu, hệ thống có thể được đóng gói bằng Docker và triển khai trên máy chủ cục bộ có trang bị GPU

của Học viện. Việc sử dụng các mô hình mã nguồn mở thế hệ mới (như Llama 3.1 hoặc Qwen) chạy nội bộ sẽ giúp hệ thống vận hành hoàn toàn độc lập, ổn định 24/7 và sẵn sàng phục vụ lượng truy cập khổng lồ trong các mùa tuyển sinh cao điểm mà không phụ thuộc vào chi phí API bên ngoài.

KẾT LUẬN CHUNG

Đề tài *"Hệ thống Chatbot tư vấn tuyển sinh"* ứng dụng kiến trúc RAG nâng cao đã hoàn thành trọn vẹn các mục tiêu nghiên cứu và phát triển được đặt ra ban đầu. Qua

quá trình phân tích, thiết kế và triển khai thực nghiệm, hệ thống đã chứng minh được tính khả thi cùng sự ưu việt của việc ứng dụng trí tuệ nhân tạo tạo sinh vào thực tiễn công tác tư vấn giáo dục tại Học viện. Thành công của dự án không chỉ nằm ở việc xây dựng một công cụ tương tác thông minh mà còn ở việc làm chủ các công nghệ mới nhất trong lĩnh vực xử lý ngôn ngữ tự nhiên.

Về mặt kỹ thuật, đề tài đã xây dựng thành công một cơ sở tri thức nội bộ vững chắc, được chuẩn hóa và phân cấp từ các nguồn thông tin chính thống của Học viện. Việc áp dụng chiến lược ParentDocumentRetriever kết hợp với mô hình nhúng chuyên biệt cho tiếng việt đã giải quyết triệt để bài toán cân bằng giữa độ chính xác khi tìm kiếm và sự toàn vẹn của ngữ cảnh. Điểm sáng lớn nhất của hệ thống là việc tối ưu hóa luồng RAG truyền thống thành kiến trúc RAG nâng cao thông qua đồ thị LangGraph. Với khả năng định tuyến thông minh và kỹ thuật truy xuất song song, chatbot không chỉ đạt tốc độ phản hồi vượt trội nhờ mô hình Gemini 2.5 Flash mà còn thể hiện tư duy logic chặt chẽ trong việc bóc tách đa truy vấn, đảm bảo bao quát đầy đủ mọi khía cạnh thông tin từ điểm chuẩn đến quy chế tuyển sinh.

Về mặt trải nghiệm người dùng, giao diện xây dựng trên nền tảng Streamlit đã mang lại một không gian tương tác thân thiện, minh bạch và liền mạch. Hệ thống không chỉ duy trì được bộ nhớ ngữ cảnh qua nhiều lượt hội thoại mà còn tiên phong áp dụng cơ chế Human-in-the-loop, chủ động tạm dừng luồng xử lý để đặt câu hỏi ngược lại nhằm làm rõ nhu cầu của thí sinh. Sự phối hợp nhịp nhàng giữa giao diện và đồ thị điều phối đã giúp chatbot đạt tỷ lệ Zero Hallucination đối với các thông tin định lượng nhạy cảm như mã ngành và học phí. Sản phẩm hứa hẹn sẽ là công cụ hỗ trợ đắc lực, giúp giảm tải đáng kể áp lực cho ban tư vấn tuyển sinh của Học viện trong các giai đoạn cao điểm của mùa thi.

Về mặt học thuật và định hướng tương lai, quy trình xây dựng hệ thống từ khâu thu thập dữ liệu tự động đến thiết kế luồng đồ thị trạng thái phức tạp hoàn toàn có thể trở thành tài liệu tham khảo giá trị cho các nghiên cứu phát triển trợ lý ảo. Mặc dù vẫn còn tồn tại một số hạn chế nhất định về hạ tầng API miễn phí hay khả năng bóc tách bảng biểu PDF phức tạp, nền tảng kiến trúc mở của đề tài đã vạch ra một lộ trình phát triển rõ ràng. Trong tương lai, việc tích hợp thêm công nghệ đa phương tiện và triển khai đa kênh trên các nền tảng mạng xã hội sẽ giúp hệ thống trở thành một mảnh ghép quan

trọng trong bức tranh chuyển đổi số toàn diện của Học viện Công nghệ Bưu chính Viễn thông.

TÀI LIỆU THAM KHẢO

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.

[2] LangChain, "LangGraph: Multi-Actor, Stateful Built with LLMs," LangChain Documentation, 2024. [Online]. Available: https://python.langchain.com/docs/langgraph.

[3] Meta AI, "Introducing Meta Llama 3: The most capable openly available LLM to date," Meta AI Blog, Apr. 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Nov. 2019, pp. 3982-3992.

[5] Groq, "Groq LPU Inference Engine: Accelerating Real-Time AI," Groq Architecture Whitepaper, 2024. [Online]. Available: https://groq.com/technology/.

[6] Chroma, "Chroma: The AI-native open-source embedding database," Chroma Documentation, 2024. [Online]. Available: https://docs.trychroma.com/.

[7] Streamlit, "Streamlit: A faster way to build and share data apps," Streamlit Documentation, 2024. [Online]. Available: https://docs.streamlit.io/.

[8] G. Pasqualino, "Agentic RAG for Dummies," GitHub Repository, 2025. [Online]. Available: https://github.com/GiovanniPasq/agentic-rag-for-dummies. [Accessed: Feb 20, 2026].